ILLUSTRATED LEARNING BOOK | VOLUME 1

Computing Made Understandable

Algorithms



Original manga illustration: a story analogy. Formal labeled diagrams inside the book carry the technical definitions.

Turn everyday tasks into precise procedures. Trace searches and sorting, then explain the work they require.

First-year learning edition | 6 November 2026

K.I.V.A.S. Learning Lab | Foundations of Computer Science & Artificial Intelligence

Inside this book

Chapters are separate and numbered within each volume. Printed page numbers match this PDF. The PDF outline also includes teaching sections.

How to learn from this volume 3

Entering the World of Computing 4

Computational Thinking and Problem-Solving 12

Pseudocode, Conditions, Loops and Tracing 21

Linear Search 30

Binary Search 39

Sorting and Bubble Sort 50

Algorithmic Efficiency and Big O 59

Glossary and retrieval cues 68

Reading, sources and credits 70

How to learn from this volume

You do not need to know a programming language. Begin with the story, predict a result, then trace the formal example. Cover the feedback until you have written a reason. A diagram is useful when you can explain what each box, arrow and number means.

Learning move	What to do	Evidence to keep
Predict	Write the next state before reading it	Your first answer
Trace	Apply one operation at a time	Intermediate states
Explain	Name the rule that justifies the change	A sentence or labeled diagram
Transfer	Change one input or boundary case	A new trace
Retrieve	Return later without the solution	A corrected explanation

Conventions used throughout the set

Indices start at zero. Arrays use length for current entries and capacity for reserved slots. Binary search uses inclusive low and high bounds and the lower midpoint. A missing result is distinct from index zero. Tree height counts edges. Campus graph edges are two-way unless explicitly stated. ReLU(z)=max(0, z); the step rule includes zero when written z>=0. A raw neural score has only the meaning the task gives it. Illustrative data are not reported experiments.

Your route

This first volume builds instructions, tracing, searching, sorting and cost reasoning. Continue to Volume II for the storage structures those procedures operate on.

A USEFUL STUDY RHYTHM

Try one or two sections in a 25-40 minute session. Finish with a small new example. After a day, repeat the chapter lab without looking at its solution. The separate Test Bank supplies unfamiliar cases, and the Learning Materials companion gives optional visual and interactive routes.

Artwork follows the supplied purple, black, white and violet theme. Manga scenes are narrative analogies. Labeled technical figures and worked tables define the algorithms and numerical examples. Essential information is also stated in text.

ALG-CH01 / CHAPTER 01

Entering the World of Computing

BY THE END

Describe a task using inputs, actions, state and outputs. Write instructions a literal helper can test.

A second helper can reproduce a task when its starting conditions, allowed actions and required result are explicit. A trace preserves the intermediate evidence that a final answer hides.



ART-ALG-CH01 A literal instruction can move a jar instead of its contents. Identify the missing actions before reading the first procedure.

1.1 Putting the lid on lunch

“Put the peanut butter on the bread.” Pip, the little delivery robot at the K.I.V.A.S. Learning Lab, sets a closed jar on a slice and steps back. Look at the result before you tell Pip what went wrong. Was the jar in the wrong place? Or did you mean something quite different by the words “peanut butter”?

A person filling in the missing steps uses experience. Pip has a small menu of possible actions. It can lift a lid, scoop a spoonful, and spread the spoonful on a slice. An instruction becomes usable when you can connect its words to actions the machine already knows. This is the first design decision in any algorithm, a sequence of precise steps for solving a problem.

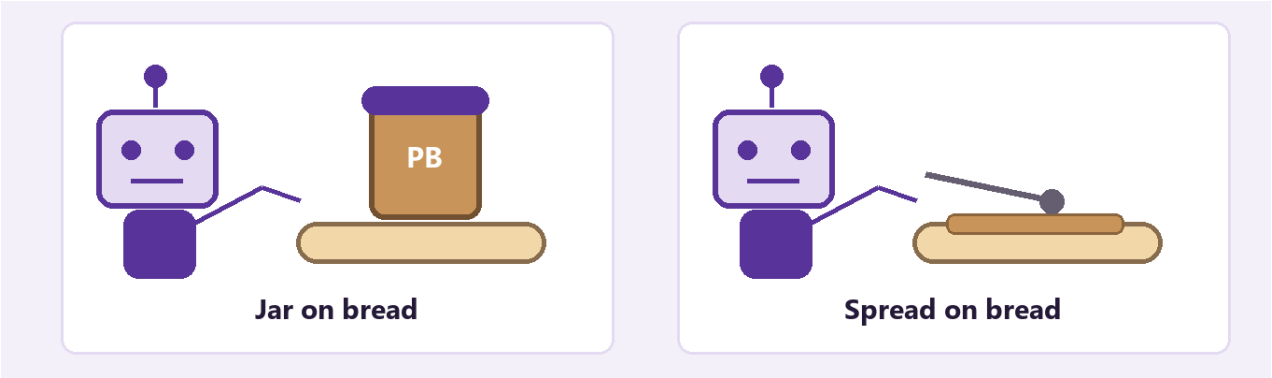


FIG-ALG-01-01 Two readings of one request. In the left drawing the whole container has moved. In the right drawing only its contents have moved. The desired end state tells us which actions to request.

Look closer: A spoonful is a useful unit only if both helpers know what to use as a spoon. Try replacing "some" with an action Pip can recognize. Precision usually means agreeing on a small vocabulary, rather than adding a long string of adjectives.

What did the robot actually do?

Suppose the jar is open, the spoon is clean, and the bread is on a plate. Cover the action column below. Say aloud what Pip should do first. Then decide which part of your instruction tells it to stop.

Step	Action	Visible result
1	Dip spoon into spread	Spoon holds one spoonful
2	Lift spoon to slice	Spread is over bread
3	Spread across top	Top has a thin layer

Reading the result: "Prepare lunch" names the goal but leaves the work to the listener. Each action in the trace has an observable effect. If the spoon were still empty after the first step, the remaining instructions could run perfectly and produce bare bread. Checking a middle step helps you locate the fault.

Try it yourself: How would you repair the plan for a sealed jar?

Check your reasoning: Add "remove the lid" before dipping the spoon. Removing it after lifting the spoon comes too late. Starting conditions determine which sequence will work.

Another case: The knife in the drawer

Your helper can open the drawer, take one utensil, close the drawer, and spread butter. The knife is inside a closed drawer and the butter is already on the bread. Write the shortest safe sequence. Which instruction relies on a fact about the starting state?

Feedback: Open the drawer, take the knife, close the drawer, then spread the butter. The final action relies on butter already being on the bread. If that fact changes, the procedure needs a scooping step. “Take one utensil” also needs a way to select the knife if the drawer holds several utensils.

1.2 A parcel comes through the hatch

Pip's next job begins with two slips in the return hatch. One says “book 18”; the other says “shelf B.” Which slip came from the visitor? Which did the desk produce? You can describe a complicated machine surprisingly well by first drawing a boundary around one job.

An input is information the job receives. An output is a result it produces. Between them lies an operation, an action that reads or changes something. A lookup operation can read a shelf label without changing the catalogue. Taking a book off that shelf changes the world even if the printed label stays the same.



FIG-ALG-01-02 Follow one request across the desk. Each arrow passes information to the next step. The middle box needs a catalogue that already connects book 18 with shelf B.

Look closer: Imagine moving the boundary from the clerk to the entire library. The shelf label was an output from the lookup. For the person carrying the book, that same label is now an input. An item's role depends on where you draw the job boundary.

Separate what arrives from what changes

The desk receives a return for book 18. Its loan card says “on loan.” A clerk changes the card and prints a receipt. List everything a second clerk would need to know to reproduce the result.

Boundary	Before	After
Message	Return book 18	Receipt for book 18
Card 18	On loan	Available
Shelf label	B	B

Reading the result: The receipt can leave the desk while the updated loan card stays inside it. An output and a stored change can occur in the same operation. Keeping both in your account matters later, when a second visitor wants the same copy.

Try it yourself: Would a missing catalogue entry prove that a book does not exist?

Check your reasoning: It proves only that the lookup found no matching entry in this catalogue. A useful response would ask the clerk to verify the identifier or check the physical copy.

Another case: A receipt without a return

A visitor scans an ID and receives a receipt. A second visitor immediately tries to borrow that copy and the desk says it is still on loan. Identify one observation that would distinguish a printing fault from a missing state change.

Feedback: Read the loan record after the first scan. If the receipt correctly names the returned copy while its record still says on loan, printing alone cannot complete the return. The record update needs attention. If the receipt names a different copy, trace how the scanned identifier reached both operations.

1.3 Snapshots of a changing desk

A visitor sees three books stacked on the return cart. A minute later there are two. Can you tell which book left? Counting captures one aspect of the situation and discards the titles. We choose what to remember by asking which questions we will need to answer.

The information a process holds at a particular moment is its state. A trace is a sequence of states recorded as instructions run. Put your finger on a row, perform exactly one instruction, and write down the next row before continuing. That tiny pause is how you catch mistakes that disappear in a long description.

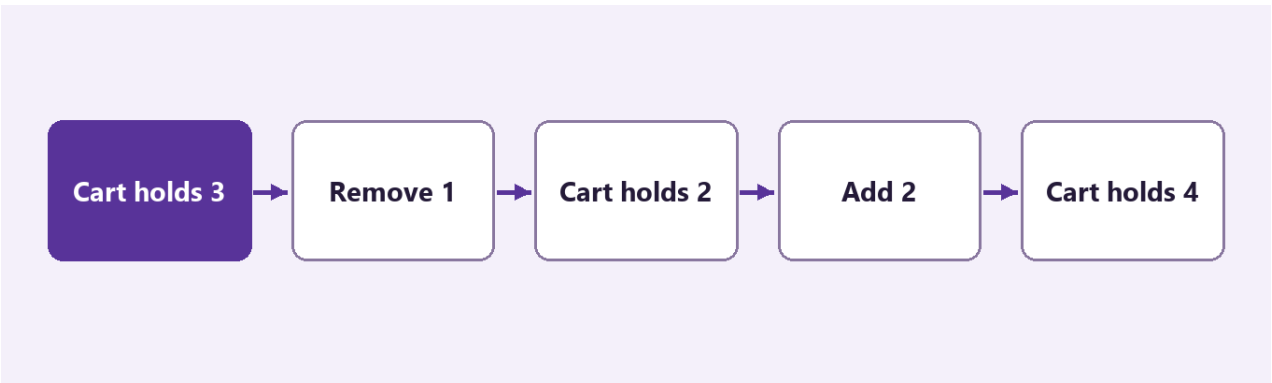


FIG-ALG-01-03 Only the quantity is recorded here. The arrows move forward in time. Reading the end first tells you the result but hides which operation caused each change.

Look closer: When reconstructing a mistake, retain the first wrong row. Crossing it out destroys evidence of the instruction that failed. Draw the intended row beside the observed one and look for the earliest pair that disagrees.

Find the first impossible snapshot

The morning cart starts with four books. Two arrive together, one leaves, and three arrive. A colleague records the following log. Recompute each row from the last state that you trust.

Event	Clerk's count	Your check
Start	4	4
Add two	6	4 + 2 = 6
Remove one	4	6 - 1 = 5
Add three	7	5 + 3 = 8

Reading the result: The first disagreement is the removal. The final row is consistent with the clerk's mistaken previous number, so blaming the last addition would repair the wrong step. In larger programs, finding the first disagreement can save hours of searching.

Try it yourself: Two carts both end at eight. Must their traces agree?

Check your reasoning: No. One could start at seven and gain one, while another starts at ten and loses two. A final state can come from several histories.

Another case: A zero that hides activity

Start with an empty tray. Two books arrive and both are immediately collected. Another tray has no events. Compare their starting and ending counts. What single additional number could distinguish these two histories?

Feedback: Both start and finish at zero. A cumulative count of arrivals distinguishes them: two on the first tray, zero on the second. A system that needs to account for work completed during the day needs a different state variable from the number currently waiting.

1.4 One method, two machines

Imagine asking two helpers to count returned books. One moves each counted book to an empty tray. The other marks each title on paper. Can they be following the same method? What matters is whether each book contributes exactly one to the final count.

A program expresses a method in instructions a particular computer can execute. The underlying algorithm can survive translation into another language or into actions with cards. A translation deserves checking. "Count everything on the tray" might accidentally include the marker that tells a machine where to begin.

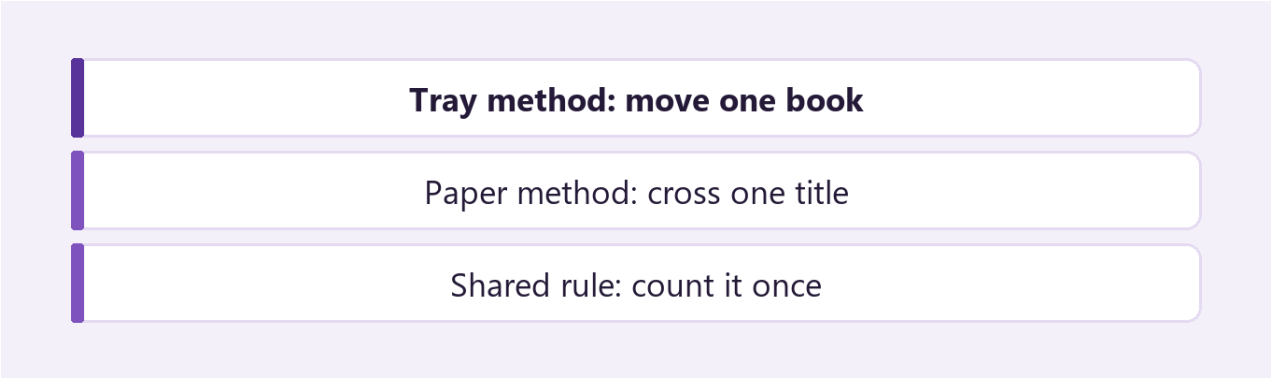


FIG-ALG-01-04 Two physical arrangements implement the same counting rule. Identify the evidence each one leaves behind that an item has already been counted.

Look closer: Ask a partner to count two red slips and one blue slip without saying the colours aloud. If their method still works, colour was incidental to the counting rule. Counting only red slips would be a different specification.

Translate a procedure without changing it

Place three cards marked Oak, Elm, and Ash on the table. Start a tally at zero. Count by moving each card once. Restore the cards and count by making a mark beside each name. Compare after each step.

After	Moved cards	Marked names	Tally
Oak	Oak	Oak	1
Elm	Oak, Elm	Oak, Elm	2
Ash	Oak, Elm, Ash	Oak, Elm, Ash	3

Reading the result: Both runs have three items and a stopping signal. The first has an empty starting tray. The second has no unmarked names. A machine needs that signal to distinguish a finished task from a pause between items.

Try it yourself: What if two different books have the same title?

Check your reasoning: Give each copy its own row or identifier. Crossing out a title everywhere could count only one of two physical books. The method must specify what counts as one item.

Another case: Translate a stop condition

One helper turns counted cards face down. Another moves counted cards into a box. Write a stop condition for each method, then give a situation where the conditions appear to hold too early.

Feedback: Stop when no face-up cards remain, or when the starting pile is empty. If a card began face down or a card fell from the pile, a helper can stop too early. Translation preserves the counting method only when preparation also preserves the meaning of the physical markers.

1.5 Three jobs at one library desk

A student asks for book 18 and then asks what else they might enjoy. The first question has a checkable answer in the catalogue. The second depends on taste. Even an excellent catalogue cannot contain a single correct answer for every possible reader.

A data structure organizes information for operations such as lookup. An algorithm carries out the lookup. A learned model uses patterns in examples to estimate a result, such as interest in a related title. Pip can retrieve a known shelf exactly while offering a suggestion with uncertainty. Later chapters will let you trace both jobs from beginning to end.



FIG-ALG-01-05 The first two boxes use stored facts. The last box can use examples of past preferences. A reader can decline the suggestion and still collect the requested book.

Look closer: Consider a title called Home held in three rooms. A human shorthand such as “bring Home” is inadequate for choosing a physical copy. A copy identifier lets the two clerks agree on what has been requested.

Write instructions another reader can run

Design a pickup routine using slips 18/B and 24/C. Give a partner, or your other hand, only the written routine. Predict what happens for requests 24, 99 and an empty message. Record each response before checking these examples.

Request	Expected desk action
24	Return shelf C
99	Report no matching record
Empty	Ask for an identifier

Reading the result: A usable routine says to read the request, check that it contains an identifier, inspect each slip, and return the matching shelf. After the last unsuccessful check it reports absence. Allowing the helper to invent a shelf makes the output impossible to audit.

Try it yourself: Can a recommendation fill the gap when book 99 has no known shelf?

Check your reasoning: It can offer an alternative title if the visitor wants one. It cannot establish where book 99 is stored. Volume I, Chapter 2 turns these distinctions into a precise agreement about what a job must deliver.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Write a literal procedure for placing three returned books on the correct shelf. Name starting conditions, inputs, stored changes, output and one failure case. Have a partner act only on your written steps; if working alone, trace each step without supplying missing instructions.

Worked guidance

A defensible plan receives copy IDs, looks up shelves, checks that each destination exists, moves one identified copy at a time and records the move. A missing shelf must yield a stated exception. A final count alone cannot prove that every copy reached its correct shelf.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can describe a task using inputs, actions, state and outputs. Write instructions a literal helper can test. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH02 / CHAPTER 02

Computational Thinking and Problem-Solving

BY THE END

Decompose a delivery, identify constraints and show why a procedure reaches a legal end state.

A specification lets you judge competing plans. A legal move protects the constraints, and a decreasing quantity can explain why repetition ends.

2.1 The delivery agreement

Pip waits at the library with a parcel. “Get it there quickly,” says a student. Pip needs a destination, a definition of arrival, and permission to use particular doors. A route that takes four minutes through a locked room may look excellent on paper.

A problem specification states what is given, which actions are allowed, and what result counts as success. We can check many candidate algorithms against the same specification. Write the agreement before choosing a method. Otherwise it is easy to declare success simply because the machine did something plausible.



FIG-ALG-02-01 The ends of the journey specify a start and goal. The middle constraint rules out attractive shortcuts. The parcel must finish at the lab along with the robot.

Look closer: Imagine timing the robot only until it reaches the laboratory door. If the delivery specification includes handing the parcel to a person, that timer stops too soon. Choose evidence that measures the result the visitor asked for.

Decide what counts as success

Three helpers propose deliveries. The parcel must arrive at the lab by 12 noon, using open paths. Predict which proposals satisfy the agreement and underline the clause that decides each case.

Arrival	Route	Parcel
11: 55	Open paths	At lab
11: 52	Locked passage	At lab
11: 50	Open paths	Left at library

Reading the result: Only the first proposal meets every condition. Time alone cannot score this problem. Treating a missing parcel as a minor penalty changes the problem from delivery into robot travel.

Try it yourself: What should the desk return when no legal route exists?

Check your reasoning: Agree on an explicit failure result, such as "delivery unavailable." Otherwise the helper might wait forever for an answer the map cannot provide.

Another case: Two answers to a missing floor

A delivery slip gives "room 12" but two buildings have a room 12. One helper chooses the nearer room and another asks the visitor to identify the building. Which action can be justified from the original slip alone?

Feedback: The slip does not distinguish the destinations. Asking for the building preserves the original goal. Selecting the nearer room silently introduces a new rule. If the desk wants that rule, it must be agreed in advance and the resulting destination should be visible to the visitor.

2.2 Divide a delivery into jobs

A clerk can wrap a book without knowing the route to the laboratory. A navigator can find the route without knowing the title. Dividing a problem into smaller jobs is decomposition. A good division gives each helper enough information to do one job and pass a useful result onward.

The connection between two jobs is as important as the jobs themselves. If the clerk writes a student's name on the package and the navigator needs a building name, both helpers may do excellent work and the delivery will still fail.

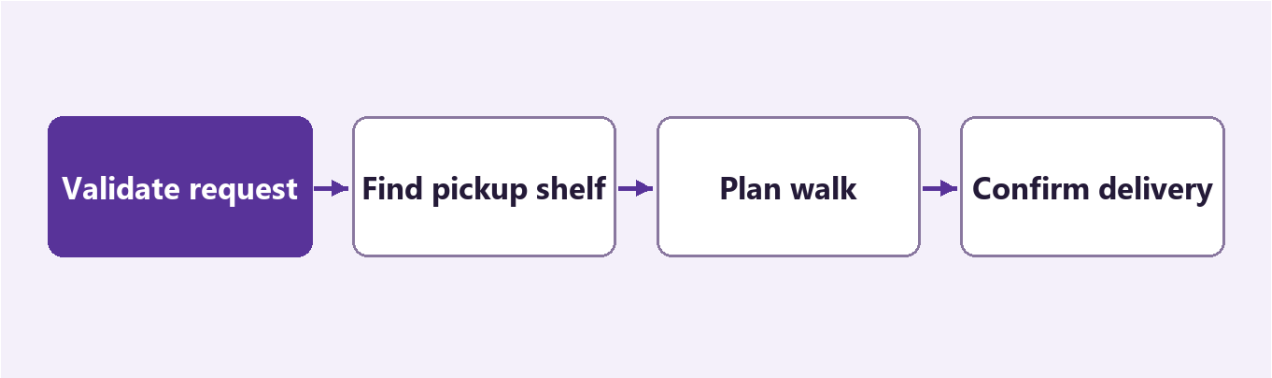


FIG-ALG-02-02 Each box answers a smaller question. Read across an arrow as “the next job receives this result.” A confirmation belongs after a completed handoff.

Look closer: Test a job boundary by handing one helper only the previous helper's output. If your navigator says “Where am I starting?”, the missing fact belongs in the handoff. Adding more detail about the book's author will not answer it.

Trace a failed handoff

Request 24 asks for pickup at shelf C and delivery to the lab. Cover the right column and supply a result that the next worker can actually use. A missing entry should stop the journey before the route planner starts.

Job	Information received	Useful result
Validate	Book 24, destination Lab	Accepted request
Catalogue	Book 24	Shelf C
Navigation	Shelf C and Lab	Open walking route
Handoff	Package and recipient	Receipt

Reading the result: The navigator needs two places. A title by itself supplies neither a reliable starting point nor a destination. Naming the information that crosses each boundary makes it possible to test each job separately.

Try it yourself: If the shelf location changes, which result should you check first?

Check your reasoning: Check the catalogue lookup. If it returns the new shelf, inspect whether navigation actually received that value or reused an old one.

Another case: A shortcut that couples two helpers

Suppose the route planner reads the current shelf by inspecting the clerk's handwritten notebook directly. Tomorrow the clerk reorganizes the pages. Explain why passing a shelf code would make a better handoff here.

Feedback: A shelf code such as B4 has a declared meaning independent of the notebook layout. Reading an undocumented page position makes route planning depend on a private arrangement. A change inside the catalogue job can then break another job even though it still returns the correct shelf.

2.3 What belongs on the map?

The same campus can have a tourist map, a wiring diagram, and a wheelchair-accessible route map. Each leaves things out. Abstraction means keeping the details that matter for a question while setting aside the rest. The test is whether the simplified description still lets you answer the question correctly.

For a dry parcel on a sunny day, a covered walkway may be unimportant. For a water-sensitive parcel in a storm it may decide the entire route. A pattern is a repeated relationship that can help us reuse reasoning. Every walkway connects two locations, even though its length and accessibility may differ.

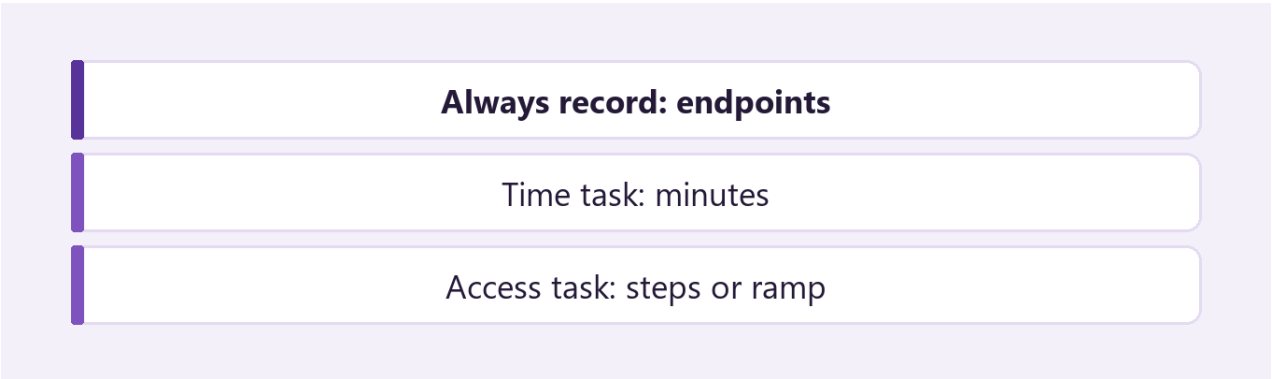


FIG-ALG-02-03 Read each line as a possible detail on the same map. The last two lines answer different requests. A map designer must know which requests to support.

Look closer: A map of time in minutes is itself a model. Walking times measured on a quiet afternoon may be poor guides during a crowd. A route can be correct for its recorded costs and still need new measurements.

Remove a detail and test the consequence

A lift takes four minutes. A stairway takes two. A ramp takes five. The lift is closed today. Compare three instructions for choosing a path. Assume Pip uses wheels and cannot use steps.

Map records	Choice suggested	Practical result
Minutes only	Stairway	Wheels cannot climb it
Minutes and access	Lift	Door is closed
Minutes, access, opening	Ramp	Legal five-minute route

Reading the result: More recorded detail is useful only when the algorithm consults it. Filter out unusable paths first, then compare travel times among those that remain. This order saves Pip from choosing an impossible "best" path.

Try it yourself: Which information could be omitted for all three routes here?

Check your reasoning: Wall colour could be omitted because none of the stated constraints uses colour. A different task, such as guiding a visitor by landmarks, might need it again.

Another case: A second map for a second visitor

A walking student wants the fastest route; a visitor on wheels needs a step-free route. The stair takes two minutes and the ramp five. Show what each planner would choose and name one detail both planners need.

Feedback: Both need the connections and their times. The walking student can use the two-minute stair under these assumptions. The wheeled visitor must first restrict the candidates to step-free edges, leaving the five-minute ramp. Adding a step-free flag makes the map support both questions.

2.4 Decisions on a delivery run

At a junction Pip sees a door. The next step depends on whether the door is open. A decision chooses an action from alternatives by checking a condition, a question with a yes or no answer. A sequence simply runs actions in a fixed order.

Repetition runs a step again while there is work to do. It needs both a condition and an action that can eventually change that condition. Waiting for a locked door to open may be an acceptable real-world policy only if someone specifies a time limit or an alternative.

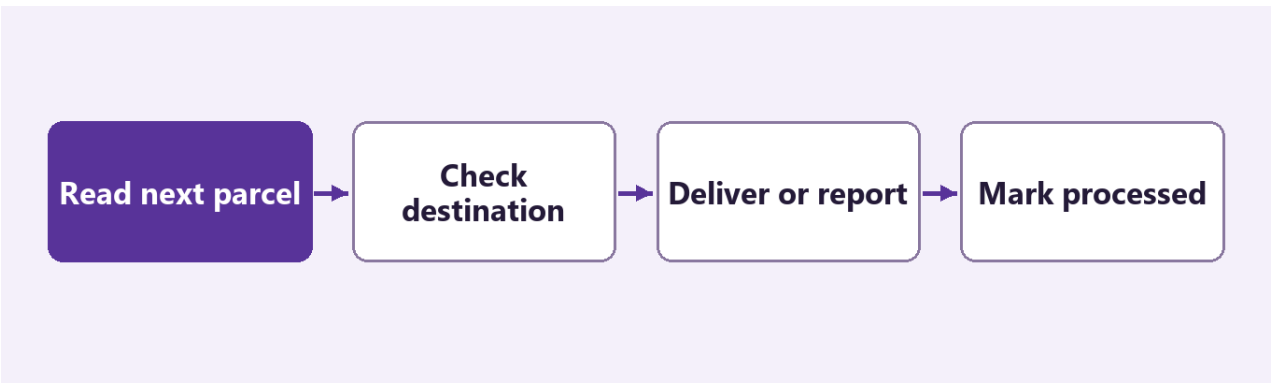


FIG-ALG-02-04 One cycle consumes one outstanding parcel, even if delivery fails. Marking a failure separately preserves the reason and lets the next parcel proceed.

Look closer: "Until all slips are dealt with" needs an observable meaning. Try making two trays, one for unprocessed requests and one for completed or failed requests. The empty first tray supplies an ending that both helpers can recognize.

Turn a rule into a complete set of cases

Draw three parcel slips labelled Lab, blank, and Gate. The rule is “while slips remain, remove the first. If its destination is blank, record a fault. Otherwise send it.” Predict the number of sends and the number of slips left.

Slip	Decision	Slips left
Lab	Send	2
Blank	Record fault	1
Gate	Send	0

Reading the result: There are two sends and one fault. Moving the removal into only the successful branch would leave the blank slip at the front forever. Drawing one row per attempt exposes this kind of mistake.

Try it yourself: Would “repeat three times” work for tomorrow's unknown pile?

Check your reasoning: It works for exactly three slips. Testing for an empty pile describes a family of inputs, including a day when there are no parcels.

Another case: An empty batch at opening time

Trace the slip-processing rule when the clerk opens an empty tray. Then compare it with the instruction “remove one slip and keep going while more remain.” Why do the two versions differ on this input?

Feedback: The condition-first version checks the tray and finishes with no removals. The second version attempts a removal before checking whether a slip exists. The error remains invisible on many ordinary days, which is why an empty input deserves a place in the test set.

2.5 A boat with one spare seat

A farmer must take a wolf, goat and cabbage across a river. Only the farmer rows, with room for one passenger or none. Without the farmer, the wolf eats the goat and the goat eats the cabbage. Try a first trip before looking at the diagram. Which pair would you leave alone?

A state records which side each traveller occupies. An action moves the farmer and at most one companion from the farmer's current shore. A legal action must leave both shores safe. A solution is a chain of legal actions from the starting state to a state with everyone across.



FIG-ALG-02-05 The fourth move in the journey reverses an earlier delivery. Moving something back can be necessary to make progress toward the complete goal.

Look closer: The boat puzzle is small enough to reset by hand. If an attempted crossing fails, note the entire state immediately before that move. You can return there and try a different legal action without repeating the whole journey.

Find a legal crossing plan

Use four marked scraps and a pencil as the river. Continue from the figure. Before every crossing, circle the shore that will lose the farmer's protection. Check that it contains neither forbidden pair.

Trip	Farmer travels with	Far bank after trip
1	Goat across	Farmer, goat
2	Alone back	Goat
3	Wolf across	Farmer, goat, wolf
4	Goat back	Wolf
5	Cabbage across	Farmer, wolf, cabbage
6	Alone back	Wolf, cabbage
7	Goat across	All four

Reading the result: The wolf can stay with the cabbage. That harmless pair is the reason the farmer can return alone on trip six. Checking only the arrival shore would miss a danger on the departure shore.

Try it yourself: Can wolf and cabbage exchange roles in this plan?

Check your reasoning: Yes. They are each unsafe only with the goat when unattended. Exchanging them in trips three and five yields another valid sequence.

Another case: A tempting first crossing

Imagine the farmer takes the cabbage first. Write the state of each shore immediately after arrival. Which observation settles whether this is a legal first move, without playing any later move?

Feedback: The far bank has the farmer and cabbage. The starting bank has wolf and goat without the farmer, so that pair is unsafe. A plan with this illegal first state is already invalid regardless of how attractive its later moves appear.

2.6 How do we know a plan finishes?

A route finder can follow perfectly legal roads in a circle for hours. Every individual move is allowed, yet the parcel never arrives. Correctness concerns the result a procedure promises for a permitted input. Termination asks whether it eventually stops. A useful procedure must address both questions.

For a pile of requests, count the remaining slips. If every step removes one and none are added during this run, that count cannot decrease forever. This argument concerns every finite starting pile. A successful run on three slips gives evidence for one case only.



FIG-ALG-02-06 The quantity above each state is a whole number. It decreases at every step. That fact would cease to hold if a failed request were silently put back at the front.

Look closer: Distinguish a failed job from a failed algorithm. A delivery algorithm that correctly reports an unreachable destination can satisfy its specification. An algorithm that claims success without arriving has failed even if it stops promptly.

Try to break the delivery procedure

Write the desk procedure for a fixed pile. Test an empty pile, a single malformed slip, and a pair with one unreachable destination. Predict the reply and remaining count for every case. Mark any step at which a retry could occur.

Input	Expected action	Remaining after run
Empty	Finish immediately	0

Input	Expected action	Remaining after run
Missing destination	Record invalid request	0
Reachable, unreachable	Deliver one and report one failure	0

Reading the result: Boundary cases test the edges of ordinary use. Invalid input breaks the stated format and deserves a defined response. A valid destination with no available route is a legitimate request with an unsuccessful outcome. Keeping these outcomes separate helps the clerk fix the right thing.

Try it yourself: What small change destroys the decreasing-count argument?

Check your reasoning: Replacing each failed slip on the pile can keep its size constant. A retry limit and a final failure response restore a route to termination. Next we will write rules like these in a compact language for people.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Plan a three-stop delivery with a wheeled robot. Mark one closed path and one stairway. Decompose the job, choose a representation and give a reason the plan terminates even when a delivery is impossible.

Worked guidance

Retain travel costs, access and closure information. Separate validating the request, selecting a feasible route, moving and confirming. A finite list of unprocessed requests can decrease; a route search needs a visited rule or another termination argument. Legal moves alone do not prove completion.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can decompose a delivery, identify constraints and show why a procedure reaches a legal end state. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH03 / CHAPTER 03

Pseudocode, Conditions, Loops and Tracing

BY THE END

Translate a rule into a branch, follow a loop and locate the first incorrect variable update.

A branch chooses one path. A loop revisits a test. Assignment changes stored state, while a comparison only reports a relationship. A trace makes their order visible.

3.1 A name for a changing number

Pip has one erasable label on the cart. It says 4. Two books arrive, so Pip reads 4, adds 2, wipes the label, and writes 6. A variable is a name for a value that can be read and changed. We can use the name count whenever we mean the current value on this label.

In pseudocode, human-readable instructions, we write “count <- count + 2.” Read the arrow as “store.” Work out the right-hand expression using the old value, then put its result under the name on the left. An equality question such as “count = 6?” checks a relationship. It leaves the label untouched.



FIG-ALG-03-01 Each box separates reading, calculating and writing. The final label replaces the earlier number. A trace table can retain a history even when the variable itself cannot.

Look closer: Read an assignment aloud backward across the arrow: “calculate the right side, then store it on the left.” On a paper trace, write the old value on one line and the new value on the next. This keeps the two moments separate.

Read the right side first

Trace the following instructions with one slip for each variable. Start with count equal to 4. Copy count to saved, add two to count, then ask whether saved equals count.

Instruction	count	saved
count <- 4	4	unset
saved <- count	4	4
count <- count + 2	6	4
saved = count?	6	4

Reading the result: The last comparison is false. Copying a number into saved took a snapshot of its value. Later changes to count have no effect on that saved number. An unset label has no value yet, so avoid reading it before the first assignment.

Try it yourself: What would count <- saved do next?

Check your reasoning: It reads 4 from saved and writes 4 into count. Both labels then hold 4, and a fresh equality check would answer true.

Another case: Swapping two labels

Start with a = 2 and b = 9. Execute a <- b then b <- a. You wanted them to exchange values. Write both intermediate states, find the lost number and repair the method with a third label.

Feedback: After the first assignment both labels contain 9. The second assignment simply copies 9 back, so the old 2 is unavailable. Save a in temp first, then assign a <- b and b <- temp. Reading the old state before overwriting it makes the exchange reversible.

3.2 A yes-or-no question with two exits

“The cart can take three books. If it holds fewer than three, accept a book. Otherwise say full.” This ordinary sentence contains a comparison and two possible results. We can draw its shape before worrying about the spelling of any programming language.

A Boolean condition has one of two results, true or false. The symbol < means strictly less than. A flowchart puts an action in a rectangle, a question in a diamond and a start or end in a rounded shape. The labels on the arrows leaving a diamond tell you which branch to follow. An input or output can be shown in a slanted box.

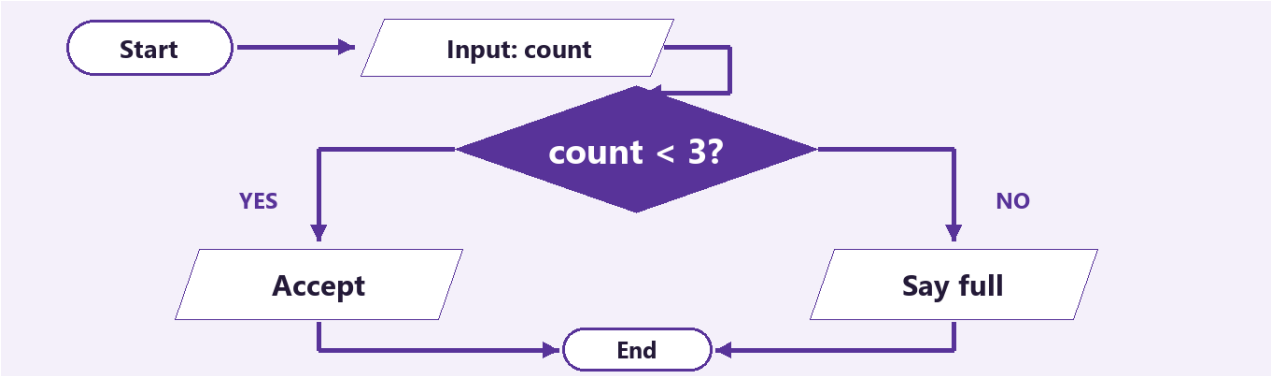


FIG-ALG-03-02 Start with the count in the top box, evaluate the diamond, then follow just one labelled arrow. Both branches finish the same decision.

Look closer: Cover the labels on the two exit arrows. Now you can see there are two choices, but you cannot tell which one follows a true answer. Yes and no carry essential information even in a beautifully drawn flowchart.

Read one rule three ways

Read this pseudocode as the same cart rule. INPUT count receives the current number. IF count < 3 THEN OUTPUT “accept” ELSE OUTPUT “full”. OUTPUT reports a result. This rule decides whether admission is allowed. A separate action would load the book.

count	count < 3?	Output
2	True	Accept
3	False	Full
0	True	Accept

Reading the result: Using ≤, meaning “less than or equal to,” would admit another book when all three spaces are occupied. Test the exact boundary as well as one value on either side. Assume the input has already been checked to be a whole number from zero to three.

Try it yourself: How does count > 2 behave on these valid inputs?

Check your reasoning: It is true exactly for 3. It can be used for the same decision if the true branch says “full” and the false branch says “accept.”

Another case: A different capacity

Change the cart's capacity from three to five. Predict the responses for counts 4 and 5. Then say what should happen before the decision if the input says -1.

Feedback: The rule count < 5 accepts 4 and reports full at 5. A negative count lies outside the agreed input range. A validation step should flag it before testing capacity; otherwise -1 < 5 would give an apparently valid acceptance for an impossible state.

3.3 A decision inside a decision

A locked lab door has a reader for a card. If there is no card, Pip asks for one. If a card is present, Pip checks its permission. Trying to inspect permission before obtaining a card would ask a question with no object.

A nested decision is one decision placed within a branch of another. Indentation makes the nesting visible in pseudocode. One decision can protect a later operation by making sure its required information exists. Trace only the branch selected at each step.



FIG-ALG-03-03 This drawing follows the successful path. Two early exits are possible: ask for a card at the first question or refuse entry at the second.

Look closer: Indentation is easiest to test by colouring matching branches with a pencil. Mark the question each ELSE answers. A missing card should never lead to an attempt to inspect a permission printed on that card.

Explain a path that never runs

Write a two-level rule on paper. Start with IF card is present. Inside the true branch add IF card allows entry THEN unlock ELSE deny. The outer ELSE asks for a card. Match each message to just one outcome.

Input	Questions reached	Response
No card	Presence only	Ask for card
Visitor card	Presence and permission	Deny
Lab card	Presence and permission	Unlock

Reading the result: The visitor and lab cards take the same first branch. Their results diverge at the second question. If “ask for card” is attached to the inner ELSE by mistake, a visitor who already showed a card will be asked for another one.

Try it yourself: Where would you add a record of every attempt, including missing cards?

Check your reasoning: Place it before the outer decision or after all branches rejoin, provided no branch exits before that record. Putting it only inside the true branch omits the missing-card case.

Another case: Two branches give the same message

Change the access rule so both a missing card and a denied card display “ask at desk.” Does identical displayed text prove that the paths taken through the procedure were identical? Give evidence you could record.

Feedback: The same final text can come from different checks. With no card, the permission check never runs. With a denied card, it runs and fails. A trace with columns for presence and permission, including “not reached,” distinguishes the paths without exposing unnecessary personal details.

3.4 Repeating a known number of times

Pip wants the total weight of three packages. It could name all three additions in advance, but tomorrow there might be four packages. A FOR loop describes one visit for each item in a specified collection. The loop body contains the actions repeated on each visit.

An accumulator is a variable that carries a running result. Set total to zero before the first visit. On each visit, add the current weight to the previous total. Choosing zero matters because it leaves the first weight unchanged when added. The same instruction can now handle an empty tray.



FIG-ALG-03-04 Each box records the state after one package, except the first box, which records preparation. The total can be read after the last visit.

Look closer: Try starting the accumulator at one instead of zero. On [2, 4, 1] the result becomes eight. The extra unit came from the initial state rather than any parcel. Testing an empty tray exposes that unwanted contribution immediately.

Count visits and additions separately

Trace “total <- 0; FOR each weight in [2, 4, 1]: total <- total + weight; after the loop OUTPUT total.” Square brackets enclose a sequence. Suppose another learner moves the initialization inside the loop. Compare the two traces.

Weight	Correct total	Reset inside each visit
2	2	2
4	6	4
1	7	1

Reading the result: Resetting inside loses the earlier work. The faulty result looks plausible if there is only one parcel. Testing three different weights makes the fault visible. The counterexample matters more than the number of successful one-parcel tests.

Try it yourself: What does the correct procedure output for no packages?

Check your reasoning: It reports zero. The initialization still runs, the body runs zero times, and the output reads the initialized total.

Another case: A constant added in the wrong place

A parcel's packaging adds one unit to each recorded weight. Compare adding one to the final sum with adding one on every visit for weights 2, 4 and 1. Explain the difference in words.

Feedback: The contents weigh seven. One extra unit at the end gives eight and represents a single shared package. Adding one on every visit gives ten, representing three individual packages. The placement of the update expresses a real assumption about what is being weighed.

3.5 Repeating until a condition changes

A cart waits while parcels remain. Tomorrow's parcel count may be unknown when the instructions are written. A WHILE loop tests a condition before each possible pass. If it is already false, its body never runs. If it stays true forever, the loop never finishes.

Set remaining to the number of parcels. While remaining is greater than zero, process one parcel and subtract one from remaining. A trace needs one more condition check than successful processing steps, because a final false check is how the machine discovers that it is done.

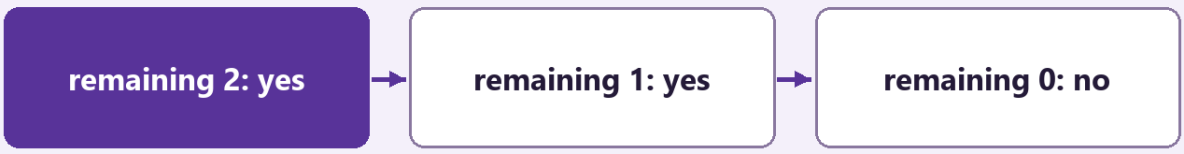


FIG-ALG-03-05 Yes admits one visit to the body. No sends control past it. The zero state is checked and left alone.

Look closer: A loop can terminate too early as well as too late. A condition remaining > 1 would stop with a parcel still on the cart. Watch both the last visit that occurs and the first visit that is skipped.

Catch a loop stuck at one

Start with two slips and put a pencil tip on the condition. Move it to the body only when the condition holds. Compare with a faulty version that subtracts one only when remaining exceeds one. Where do the two runs first diverge?

Check	Correct remaining	Faulty remaining
1	2	2
2	1	1
3	0, exits	1, repeats
4	Finished	1, repeats

Reading the result: The faulty run keeps seeing a true condition but loses the update that would make it false. Stopping a paper trace when a complete state repeats can expose such a loop. In this example no other input changes between checks.

Try it yourself: Would changing the original condition to remaining ≥ 0 fix anything?

Check your reasoning: It adds an unwanted iteration at zero. Pip would try to process a parcel that does not exist and reduce the count to -1.

Another case: A supply that keeps arriving

A worker starts with two parcels. After every removal a new parcel arrives. Apply the rule “while parcels remain, process one.” Which part of the earlier termination argument no longer works?

Feedback: The number waiting stays at two after each complete remove-and-arrive cycle. It no longer decreases toward zero. To process a finite batch, separate the two initial parcels from incoming work or stop after the agreed batch size. This changes the specification in a way the team can explain.

3.6 A trace you can defend

A delivery desk wants to know how many parcels weigh more than three units. This combines everything so far. It has an input sequence, a loop to visit parcels, a condition to select some of them, and an accumulator to count the selected parcels.

First say the rule in ordinary language. Start a tally at zero. For every weight, add one mark only if that weight exceeds three. Report the tally after the final parcel. This order matters: a question about one parcel belongs inside the loop, while a question about the whole tray usually belongs after it.

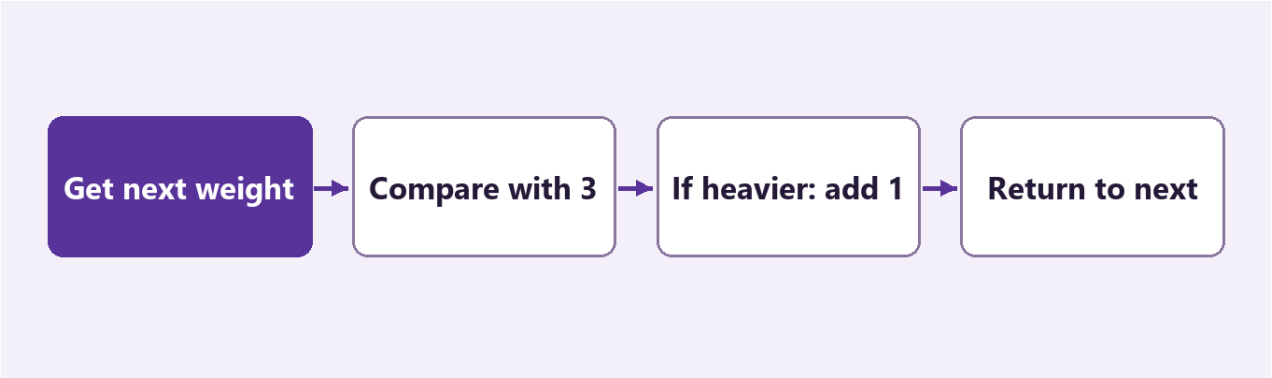


FIG-ALG-03-06 A failed comparison bypasses the increment. Both outcomes lead to the next weight until the tray is exhausted.

Look closer: Counting heavy parcels adds one per qualifying parcel. Finding their total weight adds the weight instead. The loop and comparison can be identical in both procedures; the right-hand side of the update determines what the final number means.

Repair the parcel counter

Use slips 3, 5, 2, 6. Predict the count without arithmetic, then write pseudocode from the paragraph. Run the instructions with a tally slip. Record the weight, comparison and tally after every visit. Test a second version that uses ≥ instead of >.

Weight	Weight > 3?	Tally
3	No	0
5	Yes	1
2	No	1
6	Yes	2

Reading the result: The first program reports two. The altered program also counts the weight equal to three and reports three. To diagnose a mismatch, compare the rows at the first weight before checking the final total. A precise comparison gives a precise meaning to “heavy.”

Try it yourself: If the request changes to “identify the first heavy parcel,” what could you stop doing?

Check your reasoning: Stop visiting parcels as soon as the first qualifying one is found, and report its position. Volume I, Chapter 4 develops exactly this idea of early success.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Trace total initially 0 over [2, 4, 1]. Then move total <- 0 inside the loop and trace again. Explain the first moment the two programs diverge. Include an empty input.

Worked guidance

The correct totals are 0, 2, 6, 7; the reset version ends at 1. At the second iteration resetting discards the previous 2. On an empty input the outside initialization provides total 0; a version initialized only inside the loop may have no defined result.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can translate a rule into a branch, follow a loop and locate the first incorrect variable update. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH04 / CHAPTER 04

Linear Search

BY THE END

Return a first match or a distinct missing result and account for every inspected position.

Each failed comparison eliminates one position. A match may stop a first-occurrence search early, while absence on an unordered row generally requires exhausting every candidate.

4.1 The mixed stack of library cards

Pip finds five catalogue cards on the floor. Someone has shuffled them. One belongs to book 7. If the top card says 4, what can Pip learn about the other four? Almost nothing. In this situation a sensible method is to turn over one card at a time.

Linear search inspects items in sequence until it finds a match or runs out of items. The number being sought is the target. To name a position, we use an index. Our first position has index zero, so the fourth position has index three. Keep “how many cards I inspected” separate from “which position I reached.”



FIG-ALG-04-01 The outlined cell is the first 7. The small numbers are indices. Reading left to right, three failures precede the success; the final 1 stays unexamined.

Look closer: You may physically flip a card or use a pencil to uncover it. Both methods model the same information restriction: Pip can act only on values it has inspected. Knowledge you gained while preparing the deck belongs to the experimenter.

Locate the first matching card

Use the Stage 1 catalogue [4, 2, 9, 7, 1]. Place an opaque slip over every value except the one being inspected. Predict the returned index and record one row per comparison. Stop immediately at a match.

Check	Index	Comparison
1	0	4 differs from 7
2	1	2 differs from 7
3	2	9 differs from 7
4	3	7 equals 7

Reading the result: The answer is index 3, reached on check 4. The algorithm need not read the last card. Its promise is to return the first matching position. Learning how many copies exist would require a different promise.

Try it yourself: What does Pip return if the target changes to 4?

Check your reasoning: Index 0 after one comparison. A zero index is a successful answer and must be kept distinct from a missing result.

Another case: The last unseen slip

Use [11, 3, 14, 6]. Before tracing, write a pair of answers for target 6: its index and the number of inspections. Then ask which part of your answer would change if a second 6 were appended.

Feedback: The first 6 is at index 3 and requires four inspections. Appending another 6 leaves both first-match answers intact because the search returns before reaching the new card. The number of matching copies changes, illustrating why the caller's request determines the stopping rule.

4.2 Where the target lands

The same stack can produce a short or a long search. Nothing about the algorithm changes when you slide a matching card from the front to the back. Only the amount of its work changes. That observation will become the foundation of efficiency analysis.

Count one inspection each time you compare a card's value with the target. Leave moving a finger and writing a tally outside this particular count. Stating what you count lets a classmate reproduce your number, even if you both take different amounts of time.

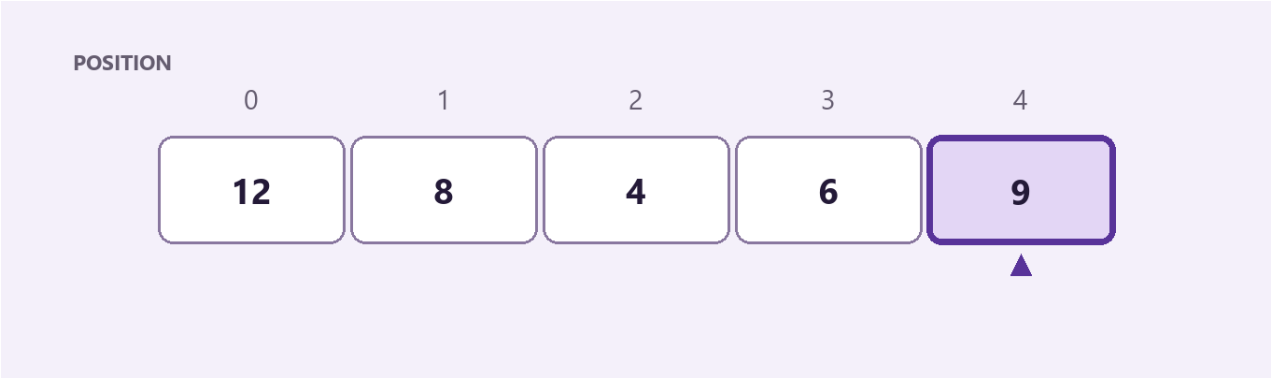


FIG-ALG-04-02 To reach 9, every card up to and including the outlined position must be read. For a unique target at index i, the count is i + 1.

Look closer: Keep the input fixed when measuring the cost of a target position. Sliding a card to the front changes the data. Changing the search procedure at the same time would make it difficult to tell which change caused a saving.

Predict the work before tracing

Use fresh copies of this row for three independent requests. For each request, predict the number of inspections, then trace from the first card to confirm. Resist starting a new request at the end of the previous search.

Request	Values visited	Count
12	12	1
4	12, 8, 4	3
9	12, 8, 4, 6, 9	5

Reading the result: The count for a middle item depends on its actual position. “Middle” is not a special operation in a linear search. The algorithm still begins at the front. A frequent request near the front can be cheap even in a large collection.

Try it yourself: If 12 is requested three times and 9 once, how many inspections are made?

Check your reasoning: Three first-position hits cost three inspections together. The last-position hit costs five more, for eight across four independent searches.

Another case: A popular title moves

On [A, B, C, D], predict the inspection total for requests D, D, A. Move D to the front while preserving the order of the other entries and repeat. Give the new total and one cost the calculation leaves out.

Feedback: Initially the three scans cost 4 + 4 + 1 = 9. The new row [D, A, B, C] gives 1 + 1 + 2 = 4. The comparison ignores moving the entries and any new costs for other visitors. A workload-specific improvement is not a universal improvement.

4.3 Absence needs evidence

A visitor asks for card 5. Pip looks at 4, then 2, then 9. A careless helper says “9 has passed 5, so give up.” That shortcut assumes a sorted row. The original cards fell on the floor. A smaller card can still be under the next cover.

To report absence from an unordered sequence, eliminate every position. After k failed checks the first k positions are known to differ from the target. This fact, true throughout the search, gives meaning to the work already done. An unchecked card is still a candidate.

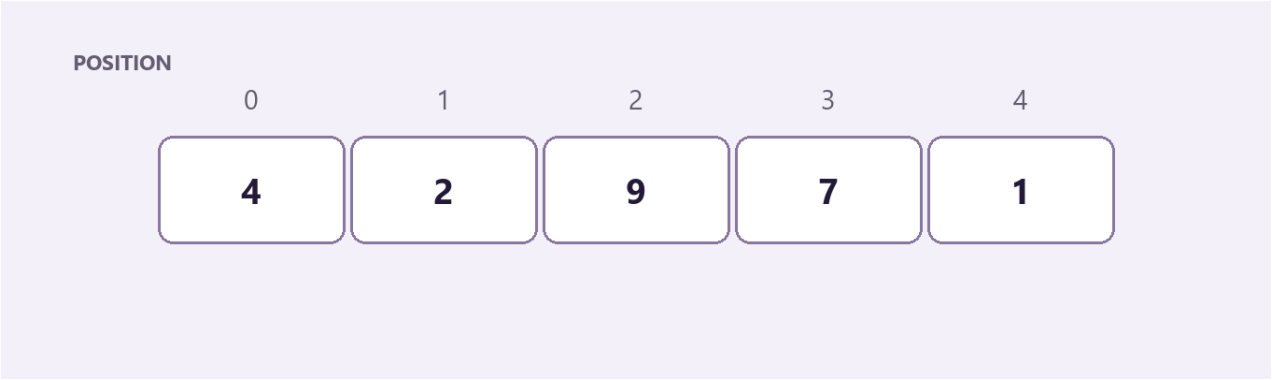


FIG-ALG-04-03 After the last failure the row contains no candidate. All five positions must contribute evidence before the caller receives a missing result.

Look closer: A missing report depends on a stable collection for this run. If someone quietly inserts the target into an already-checked part of the pile, your recorded failures describe an older version. Real catalogues must decide which version a lookup reads.

Decide when to say missing

Draw five small boxes for the five cards. Mark a box only after its card has failed the comparison with 5. Now imagine an interruption just after the third mark. What can you safely tell the visitor at that moment?

Checked prefix	Unexamined values	What is established
4, 2	9, 7, 1	Two early failures
4, 2, 9	7, 1	Three early failures
All five	None	5 is absent

Reading the result: “Still looking” and “missing” have different meanings. An incomplete search cannot justify the second message. If the search reports absence early, create a counterexample by replacing an unchecked value with 5. Every observation made so far would be identical.

Try it yourself: How many value comparisons establish absence from an empty row?

Check your reasoning: Zero. There is no candidate to eliminate. The procedure still performs the check that determines the row is empty.

Another case: Same observations, different answer

A helper has uncovered 3, 1, 9 at the front of a five-card row and reports that 6 is missing. Construct two possible endings that agree with every observation but require different final answers.

Feedback: End the row with 4, 2 for a miss or with 4, 6 for a hit at index 4. Both rows generate exactly the observations made so far. This pair of counterexamples shows why no inference about the unchecked suffix can justify the early missing report.

4.4 Two copies, two different questions

Two copies of book 7 can sit on different shelves. “Find a copy” and “find every copy” sound similar at the desk. They authorize different stopping rules. Decide what the visitor needs before choosing the output of your search.

For the first copy, scan left to right and return at the first equality. For every copy, keep a collection of matching indices and continue through the last position. The second method can require more storage as well as more inspections. A count-only request can use a single tally.

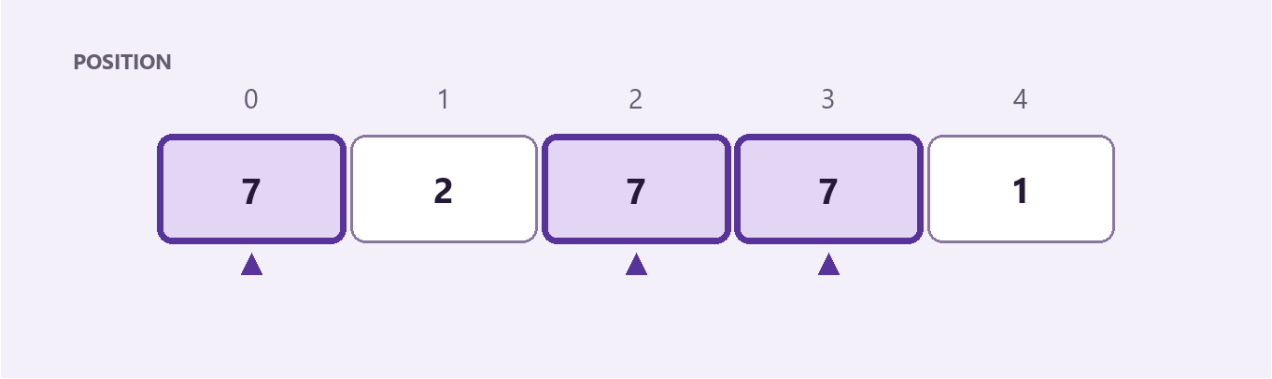


FIG-ALG-04-04 The same identifier appears at three distinguishable positions. The outlines show every match. A first-match search would stop at the first outlined cell.

Look closer: The number of matches may be zero even when the sequence contains the number zero. Name these facts carefully. “Found value 0,” “found at index 0,” and “found 0 matches” answer three different questions.

Decide which match to report

Make three copies of the request “7” and label them first, count, and all positions. Start at zero on the same row for each request. Write the smallest useful output for each visitor and the number of comparisons you needed.

Request	Output	Inspections
First	Index 0	1
Count	3	5
All positions	[0, 2, 3]	5

Reading the result: Returning 7 repeats information the visitor already supplied. A location, count, or list of locations answers a new question. If you keep a list of positions, preserve their order so a caller can work from the front of the catalogue.

Try it yourself: Would reporting index 3 satisfy "find any copy"?

Check your reasoning: Yes, because position 3 contains 7. It would fail a promise to find the first copy. Correctness depends on the exact wording of the request.

Another case: Which returns require a full scan?

Use [2, 8, 2, 2] and target 2. Compare a request for the first index with a request for the last index, keeping the standard left-to-right scan. Explain what you must remember for the latter.

Feedback: First index returns 0 immediately. For the last index, remember the latest matching position and continue; the saved value changes from 0 to 2 to 3. Only after reaching the end can you return 3. A later mismatch should leave the saved match intact.

4.5 Writing down the scan

Now that you can act out the search, give another helper an exact recipe. Let A be the row and n its length. A[i] means the value at index i. The valid indices run from zero through n - 1. The number n is already one position beyond the final cell.

RETURN hands a result back to the caller and ends this search. This makes it a good way to report success from inside the loop. A missing result belongs after the loop, when all candidates have been checked. The word MISSING will be our distinct failure result.

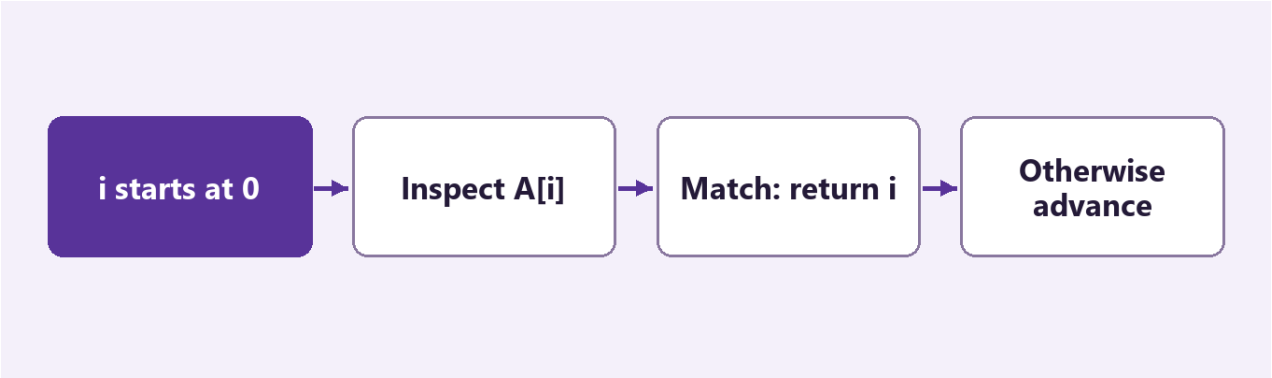


FIG-ALG-04-05 The inspection occurs only when i is smaller than n. A failed comparison advances i; the next condition check decides whether another inspection is legal.

Look closer: Take the two-line promise "return the first matching index, otherwise MISSING" and try to use it as a caller. A caller can safely ask whether it found anything before using the result as an index. Reversing those steps risks reading an invalid position.

Put the failure message in the right place

Translate this recipe into pseudocode, keeping indentation to show which actions repeat. Try it on [4, 7] with target 7. Next imagine RETURN MISSING moved just below the comparison, still inside the loop.

Line	Instruction
1	i <- 0
2	WHILE i < n
3	IF A[i] = target THEN RETURN i
4	i <- i + 1
5	RETURN MISSING

Reading the result: In the correct trace the first comparison fails and advances i to 1. The second succeeds and returns 1. An immediate failure return inside the loop rejects the request after reading only 4. A two-card example is enough to expose the fault.

Try it yourself: What happens if line 4 is removed?

Check your reasoning: A mismatch at the first cell leaves i at zero. The same comparison repeats forever. Moving to a fresh candidate supplies progress toward termination.

Another case: A caller that confuses zero with failure

A search returns index 0 for the first card. A clerk treats any numerical zero as “no result” and prints missing. Does repairing the search loop solve this failure? Identify the boundary where the information is lost.

Feedback: The search returned the promised valid index. The caller's interpretation erased the distinction between position zero and MISSING. Repair the caller to check the distinct missing marker first. A one-card hit is a small test of the entire lookup and display path.

4.6 The smallest useful test shelf

A successful demonstration on five cards leaves many questions unanswered. Does the procedure behave sensibly with no cards? Does a match at the first position count as success? Can it inspect the last position without walking beyond the row?

Good tests choose inputs because they distinguish plausible behaviours. A single matching card tests first and last position at once. A pair with the target only at the end separates an ordinary scan from a broken one that rejects after its first failure.



FIG-ALG-04-06 These tiny arrangements isolate three boundary decisions. Draw the actual cell before pretending to read it. An empty row has no position zero.

Look closer: A test that triggers a fault is most useful when small enough to explain. If a singleton exposes a skipped last cell, a thousand-entry catalogue might conceal the same cause behind pages of irrelevant comparisons.

Audit a search routine with cards

Construct the cases below with scraps. Predict the index or MISSING and the comparison count. Run the pseudocode once per case. Then replace i < n with i < n - 1 and mark every case where the result changes.

Cards and target	Correct result	Comparisons
[] seek 8	MISSING	0

Cards and target	Correct result	Comparisons
[8] seek 8	0	1
[9] seek 8	MISSING	1
[9, 8] seek 8	1	2
[8, 8] seek 8	0	1

Reading the result: The altered condition skips the last valid cell. It misses the singleton hit and the two-card final hit. For the singleton miss it still says MISSING but uses zero comparisons, so looking only at the result can conceal a fault in the trace.

Try it yourself: Which new piece of information could let Pip exclude several cards at once?

Check your reasoning: A guarantee that all cards are ordered by the search key. Volume I, Chapter 5 turns that guarantee into a different search strategy.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

For [4, 2, 9, 7, 1], trace a first-match search for 7 and for 8. Then design a count-all version for [7, 2, 7]. Distinguish the returned value from the work count.

Worked guidance

Seeking 7 returns index 3 after four inspections. Seeking 8 returns missing after five. Count-all examines all three entries and returns two matches. Stopping at the first match cannot establish the total number of matches.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can return a first match or a distinct missing result and account for every inspected position. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH05 / CHAPTER 05

Binary Search

BY THE END

Maintain inclusive bounds, justify each discarded region and decide when sorted lookup repays its preparation.

A comparison becomes powerful when a consistent sort order rules out an entire side. Every failed midpoint must leave a strictly smaller candidate interval.



ART-ALG-CH05 Read the panels left to right: an unordered pile, a useful ordering, then a lookup. This is a story overview; the numbered trace below shows exactly which side is excluded.

5.1 An alphabetized directory

Open a directory near its middle to look for Noor. If the name on the open page is Mei, would you turn left or right? Assuming the directory uses the same alphabetical order as you, Noor belongs after Mei. The unseen pages before Mei can be excluded with one comparison.

Binary search applies that reasoning to an ordered collection with direct access to a middle item. Its advantage comes from the information supplied by order. The same comparison on a shuffled collection says nothing about the side where a different item might lie.



FIG-ALG-05-01 Read the rows from top to bottom. The first midpoint, 12, rules out four low positions. The next midpoint, 18, rules out the two high ones. The only remaining candidate is 15. Crossed-out cards stay at their original indices.

Look closer: To use the directory analogy, you implicitly accept its alphabetization rules. For numeric keys printed as text, a system might put "10" before "2." Convert to a consistent numeric representation if the search compares numbers numerically.

Give a reason for discarding half

Fold a sheet over the left four values after the first comparison. Read the middle of the remaining three, then choose the only remaining candidate. Record each value and the evidence for the discarded region.

Middle value	Relation to 15	Next candidates
12	Too small	15, 18, 21
18	Too large	15
15	Equal	Return index 4

Reading the result: Sorted order justifies two different inferences. Everything at or before 12 is too small; everything at or after 18 is too large. On [3, 15, 9, 12, 6, 18, 21], the first inference would discard the actual target.

Try it yourself: Could the names be sorted by surname while the visitor searches by first name?

Check your reasoning: A midpoint comparison of first names would not justify eliminating a side. Use the same ordering key for storage and comparison.

Another case: An ordering counterexample

Binary search for 4 in [4, 9, 2, 12, 15]. Use the lower midpoint and apply the usual ascending-order updates mechanically. Which comparison first eliminates a position that actually contains 4?

Feedback: Index 2 contains 2. Since 4 is larger, the rule keeps indices 3 and 4, discarding the true match at index 0. The algorithm's arithmetic is internally consistent; the input breaks the rule that smaller keys occupy the earlier positions.

5.2 Put two fingers around the candidates

The directory is too heavy to tear in half at every decision. Keep it intact. Place one finger at the lowest candidate index, low, and one at the highest, high. Both endpoints are included. Updating a finger changes the search region while preserving every original index.

For an even number of candidates, choose the lower of the two middle positions. Compute mid = low + floor((high - low) / 2). Here floor means discard the fractional part after division. The expression measures half the gap, then steps that far from low.

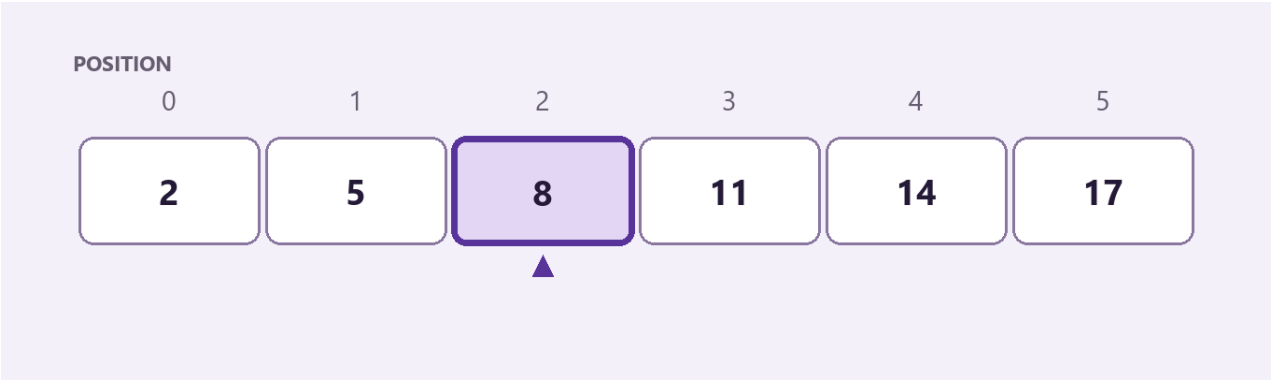


FIG-ALG-05-02 Indices 0 through 5 have a gap of five. Half the gap is 2.5. Stepping two positions from zero places the marker at index 2, whose value is 8.

Look closer: On the interval 6 through 7 the lower midpoint is 6. Make the gap calculation on paper. This two-cell example will matter when we inspect a loop that fails to move beyond the middle.

Calculate a middle position

Make a row of index labels 0 through 9. Compute the lower middle for each pair of fingers below. Check that it lands between the fingers even when both point at the same cell.

low	high	mid calculation
0	5	0 + floor(5/2) = 2
4	9	4 + floor(5/2) = 6
7	7	7 + floor(0/2) = 7

Reading the result: A common slip is to take half of high - low and forget to add low back. In the second row that would give 2, a position outside the candidate region. Checking where your finger lands is a quick way to diagnose the arithmetic.

Try it yourself: How many candidates lie from index 4 through index 9?

Check your reasoning: Six. The gap is five, but including both endpoints adds one. Candidate count is high - low + 1 whenever low ≤ high.

Another case: Local and original positions

Seven entries remain at original indices 10 through 16. A helper says “the middle is position 3.” Explain a sense in which 3 is correct and why using A[3] would nevertheless inspect the wrong record.

Feedback: There are three steps from the low endpoint to the middle. The original index is 10 + 3 = 13. The number 3 is an offset within the remaining region. Bounds measured in the original array must produce a midpoint measured in that same array.

5.3 Move beyond the middle

The middle card has just failed. Keeping it as a candidate wastes information and can prevent a small search from finishing. In an ascending row, when A[mid] is smaller than the target, move low to mid + 1. When it is larger, move high to mid - 1.

The other finger stays where it was. It still marks a valid outer limit. At each update ask two questions. Could a discarded cell contain the target? Does the remaining interval contain fewer cells? These questions address correctness and termination separately.

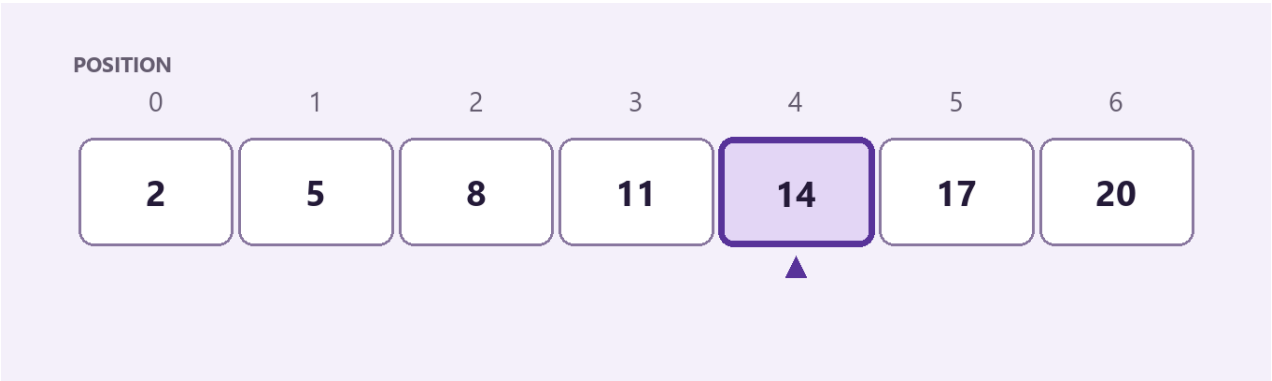


FIG-ALG-05-03 Seek 14. The first midpoint is 11, so positions 4-6 survive. The next midpoint is 17, which leaves only the outlined index 4.

Look closer: Keep low and high as indices into the original array. Copying a smaller array at each step changes the cost and loses the original offset unless you track it separately. Moving two index labels avoids both problems.

Follow both directions in one search

Copy the bounds before each comparison. Write indices and values in separate columns, especially when the target itself resembles an index. Use the lower midpoint rule throughout the book.

low...high	mid and value	Action
0...6	3 gives 11	low <- 4
4...6	5 gives 17	high <- 4
4...4	4 gives 14	Return 4

Reading the result: The final interval still contains one candidate. Testing only while low < high would skip it in this version. Endpoint conventions are a package: loop condition, midpoint and updates have to agree.

Try it yourself: What changes if the target is 17?

Check your reasoning: The first update is identical. The second comparison is an equality at index 5, so the search returns there after two inspections.

Another case: The smallest key

On [2, 5, 8, 11, 14, 17, 20], find 2. Write every pair of bounds before looking at a middle card. How many candidates survive the second failure?

Feedback: Start with 0...6 and inspect 11 at index 3. Continue with 0...2 and inspect 5 at index 1. Only index 0 survives. Its value 2 gives a match on the third inspection. In both failures the high finger moved past a known-too-large key.

5.4 When the fingers cross

What if the directory has no book 13? Binary search can prove absence without inspecting every item, because each failed comparison eliminates an entire ordered region. It finishes when low exceeds high. There is then no index that could satisfy both limits.

Initialize low to zero and high to n - 1. WHILE low ≤ high, compute mid and compare. RETURN mid on equality. If A[mid] < target set low <- mid + 1; otherwise set high <- mid - 1. After the loop RETURN MISSING. Compute mid only after checking the bounds.

Target 13 · low 4 · high 6



The middle cell is labelled “middle.” A crossed-out value has already been eliminated.

Keep lower indices | Return this index | Keep higher indices

FIG-ALG-05-04 Optional offline lab. Open Binary_Search_Lab.html in a browser after extracting the package. Choose “Seven books · find 13.” After “Keep higher indices,” the screen shows low 4, high 6 and middle value 18. Predict the next move, then inspect the history. The paper trace on the next page follows the same steps.

Look closer: A report of absence is an output about the searched row. Returning the last midpoint as a substitute for MISSING would send some callers to a nearby but incorrect record, especially for requests between adjacent keys.

Give absence a precise stopping rule

Find 13 in the pictured sequence. Work the same comparisons with counters labelled low and high. Pause before trying to read a value after the final move.

Bounds before	Midpoint	Decision
0…6	12 at 3	Raise low to 4
4…6	18 at 5	Lower high to 4
4…4	15 at 4	Lower high to 3
4…3	No midpoint	Return MISSING

Reading the result: Even though high and low now refer to real adjacent positions, their order makes the interval empty. On an initially empty collection high starts at -1, so the same guard prevents any access at all.

Try it yourself: If the target is smaller than every item, may high become -1?

Check your reasoning: Yes. It is a boundary indicating that no index remains. The guard prevents the algorithm from reading at -1.

Another case: A miss far beyond the end

Seek 30 in [3, 6, 9, 12, 15, 18, 21]. Predict the last valid cell examined and the first bounds for which the loop condition fails. Explain why no further array read is permitted.

Feedback: Read 12, then 18, then 21 at index 6. The next lower bound becomes 7 while high stays 6. The test 7 ≤ 6 fails. Index 7 is valid as a boundary marker outside the row but cannot be read as a stored item.

5.5 The search that never quite finishes

A learner proposes replacing low <- mid + 1 with low <- mid. It sounds cautious: perhaps skipping the midpoint will lose something. But the algorithm only takes this branch after comparing the midpoint and finding it too small. That position has already had its chance.

Small cases put a faulty rule under a magnifying glass. On a row of two candidates, the lower midpoint is the left endpoint. Assigning low its current value changes nothing. Repeating the comparison cannot produce new information.

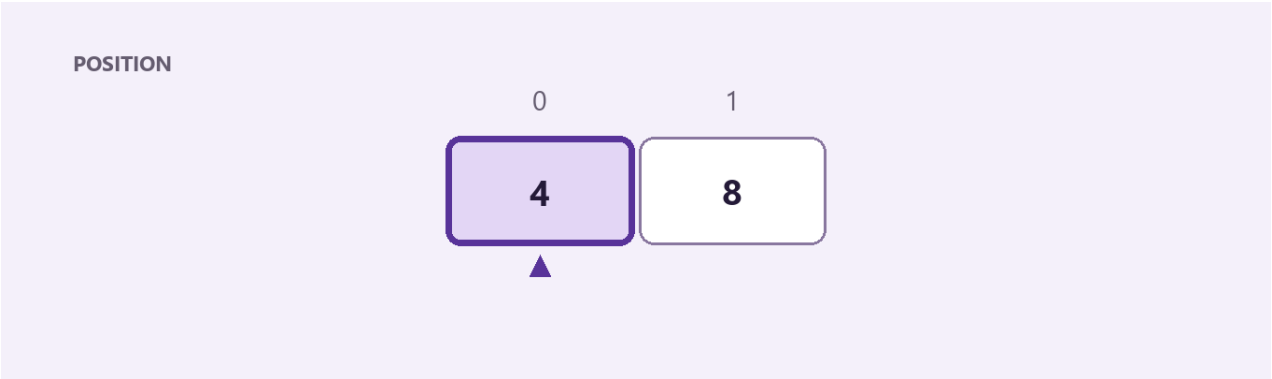


FIG-ALG-05-05 With low 0 and high 1, the lower middle is index 0. A lower bound that stays at 0 keeps proposing the same value, 4.

Look closer: An algorithm may pass tests for a middle hit while failing at an endpoint. A test that succeeds before the flawed update executes cannot detect a fault in that update. Choose inputs that actually reach the line you wish to test.

Find a two-card counterexample

Seek 8 in [4, 8]. Write two complete rows for the faulty rule and then run the repaired rule. Circle the first state that occurs twice in the faulty run.

Version	First check	Second check
Faulty	[0, 1] reads 4	[0, 1] reads 4 again
Repaired	[0, 1] reads 4	[1, 1] reads 8
Result	Faulty repeats indefinitely	Repaired returns 1

Reading the result: The example is a proof of failure for the proposed universal rule. It takes just one legal input on which the method cannot deliver. A slow successful run and a non-terminating run deserve different diagnoses.

Try it yourself: Could low <- mid be correct in some other algorithm?

Check your reasoning: Yes, with a different midpoint and stopping convention. Here the lower-middle choice and inclusive loop require a move past the failed middle.

Another case: A symmetric bug on the high side

Suppose a helper sees a too-large midpoint and updates high <- mid. Test a singleton [8] with target 4. Why does this counterexample expose the bug without requiring any calculation beyond the first midpoint?

Feedback: Low, mid and high all equal zero. Updating high to zero repeats the exact state and the value 8 is inspected again. The correct update high <- mid - 1 creates bounds 0...-1, allowing a clean missing result after one comparison.

5.6 Duplicate keys in an ordered catalogue

Sorting places equal keys together. Suppose the desk needs the first copy of book 7 in shelf order. An ordinary binary search can return any matching middle position. It has fulfilled “find a copy,” but the copies to its left have not been examined.

To seek the first occurrence, keep a separate best-so-far position. On equality, remember mid, then continue to the left by setting high to mid - 1. On ordinary mismatches make the usual updates. Return the remembered position after the interval empties, or MISSING if no match was recorded.

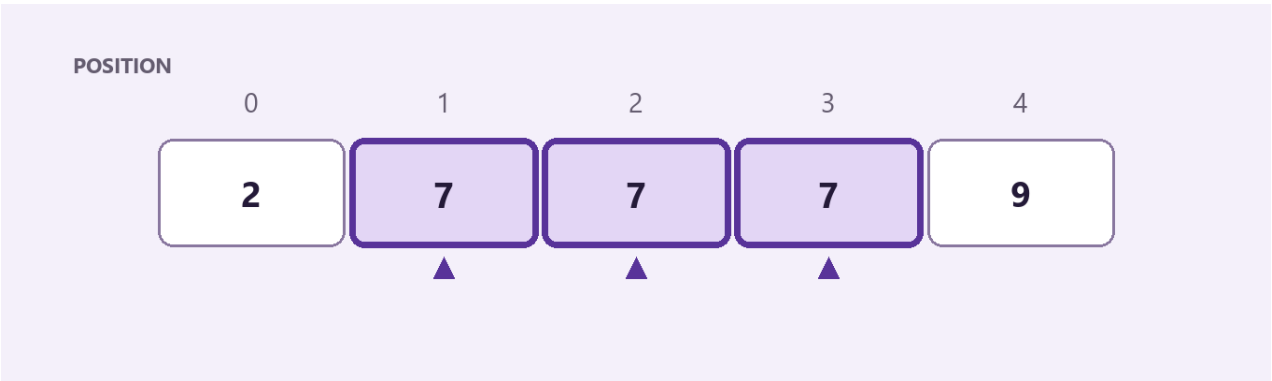


FIG-ALG-05-06 The outlined block contains three copies. The centre of the full row is a match, but the leftmost outlined index is the answer to the stricter request.

Look closer: For a first-occurrence result, earlier means smaller index, even if two physical records have the same title and edition. The saved candidate records the strongest answer established so far while the search asks whether an earlier one exists.

Find a match and then the first match

Trace the leftmost variant on these five cards. Begin with no remembered match. Make sure a later failure does not erase an earlier success.

Bounds	Middle	Remembered after step
0…4	7 at 2	2, move left
0…1	2 at 0	2, move right
1…1	7 at 1	1, move left
1…0	None	Return 1

Reading the result: Remembering index 2 before moving left matters. If index 2 had been the only match, the search to its left would have failed but the original request would still have succeeded. The saved answer preserves that evidence.

Try it yourself: How would you seek the rightmost occurrence?

Check your reasoning: On equality, save mid and continue to its right with low <- mid + 1. The other two comparison cases stay the same.

Another case: An entire row of duplicates

On [7, 7, 7, 7], seek the leftmost 7 using a saved candidate. Trace the two midpoint visits. Compare with an ordinary search and state which returned positions satisfy “any copy.”

Feedback: The lower middle on 0…3 is index 1. Save it and search 0…0, where another equality replaces the candidate with index 0. Both indices 0 and 1 satisfy any-copy search. Only 0 satisfies the stricter leftmost contract.

5.7 Should Pip sort the pile first?

The library's paper slips arrive in mixed order. A helper wants to sort them solely to run binary search once. Sorting consumes work, and reaching the middle of a physical stack may itself require handling many cards. The usual saving assumes direct access to indexed entries.

Suppose a small indexed catalogue has eight entries. For this exercise the preparation costs exactly 20 units; a value inspection costs one unit. Compare worst-case budgets of eight for a scan and four for a binary search. These are stipulated accounting units, not a measured running time.



FIG-ALG-05-07 The one-time preparation is paid only on the sorted route. Spread it across the requests made before a change forces another preparation.

Look closer: A physical card stack lacks cheap midpoint access. If reaching each midpoint requires counting through cards, counting only key comparisons hides that travel. State the organization of the collection alongside the claimed search cost.

Compare the bill for one search and many

Use counters for a batch of one, five and six requests. Predict at which batch size the sorted route gains a strict advantage in this model. Then explain how often adding new unsorted entries could change the choice.

Requests	Scan budget	Prepare and search
1	8	20 + 4 = 24
5	40	20 + 20 = 40
6	48	20 + 24 = 44

Reading the result: The budgets tie at five. At six the prepared route uses four fewer units. An actual workload with many first-position hits may favour scanning for longer. The input order, access cost and query pattern all influence the decision.

Try it yourself: Why keep the original catalogue order as well as a sorted index?

Check your reasoning: Arrival order can matter for other jobs. A second ordered list of keys and locations can support search while preserving that history, at the cost of storage and maintenance. Next we will examine what sorting actually does.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Use inclusive bounds and lower midpoints to search [3, 6, 9, 12, 15, 18, 21] for 15 and 13. Record low, high, midpoint and decision each time. Repeat with an empty row and a one-item row.

Worked guidance

For 15, midpoint indices are 3, 5, 4 and the result is index 4. For 13, inspect the same values 12, 18, 15, then low=4 exceeds high=3. An empty row starts low=0, high=-1 and uses no inspections. A one-item row still needs one comparison.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can maintain inclusive bounds, justify each discarded region and decide when sorted lookup repays its preparation. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH06 / CHAPTER 06

Sorting and Bubble Sort

BY THE END

Trace adjacent swaps, identify a settled suffix and separate comparisons from movements.

One ascending pass places a largest remaining key at the right edge. A quiet full pass establishes order. A direct exchange with a distant minimum belongs to another method.



ART-ALG-CH06 The student prepares an adjacent exchange while Pip records the steps. The loop inset reminds us to check progress as well as legal actions.

6.1 Two neighbours out of order

A cart carries numbered cards 5, 1, 4, 2. Pip wants ascending order, smallest first. If only two neighbouring cards can be handled at once, which first move would help? Compare 5 with 1. Their order is reversed, so exchange them.

A swap moves both values to each other's places. If you overwrite the first slot with the second before saving the first value, one value is lost. A spare hand or temporary label solves that problem. Sorting must preserve every original item, including repeated ones.

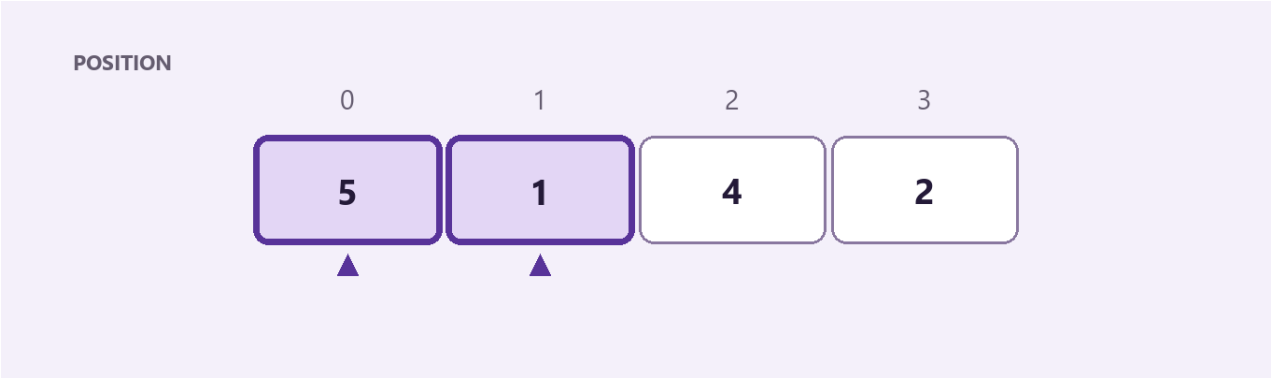


FIG-ALG-06-01 Only the outlined adjacent pair may exchange places in this step. The result is [1, 5, 4, 2]. The two uninspected cards stay where they are.

Look closer: Check two properties of a sorted result: the order of its keys and the collection of records it preserves. A machine that replaces every key with zero produces a nondecreasing row yet destroys most original records.

Make the smallest possible repair

Use a spare slip called temp to swap the first two positions. Treat "copy" as reading a value and writing it elsewhere. Compare with a version that simply writes A[0] <- A[1] and then A[1] <- A[0].

Step	A[0]	A[1]	temp
Before	5	1	unset
Save A[0]	5	1	5
Copy A[1] to A[0]	1	1	5
Copy temp to A[1]	1	5	5

Reading the result: The two slots briefly hold identical values, but the saved value survives on the temp slip. The faulty version ends with two ones. Checking only that the result increases would miss the lost card.

Try it yourself: What should happen to adjacent values 4 and 4?

Check your reasoning: Leave them in place under the strict greater-than swap rule. Their relative order already satisfies ascending order.

Another case: Protect each record

Before sorting [4A, 1B, 4C], write the three letter labels on a separate slip. A faulty sort returns [1B, 4A, 4A]. Show why checking only numeric order or the sum of keys would accept the faulty result.

Feedback: The keys 1, 4, 4 are ordered and sum to nine, just as the input did. Yet C has vanished and A occurs twice. Check record identities or the count of each full record. Sorting must preserve the original collection as well as arrange its keys.

6.2 One complete pass along the cart

Bubble Sort walks along a row comparing neighbours, swapping when the left value exceeds the right. One complete walk through the current unsorted region is a pass. After each comparison, the larger of the two occupies the right-hand position of the pair.

This gives the first pass a surprising guarantee. The largest value seen so far travels along with the inspection. By the final comparison, a largest value of the full row has reached the last slot. The rest of the row can still contain disorder.

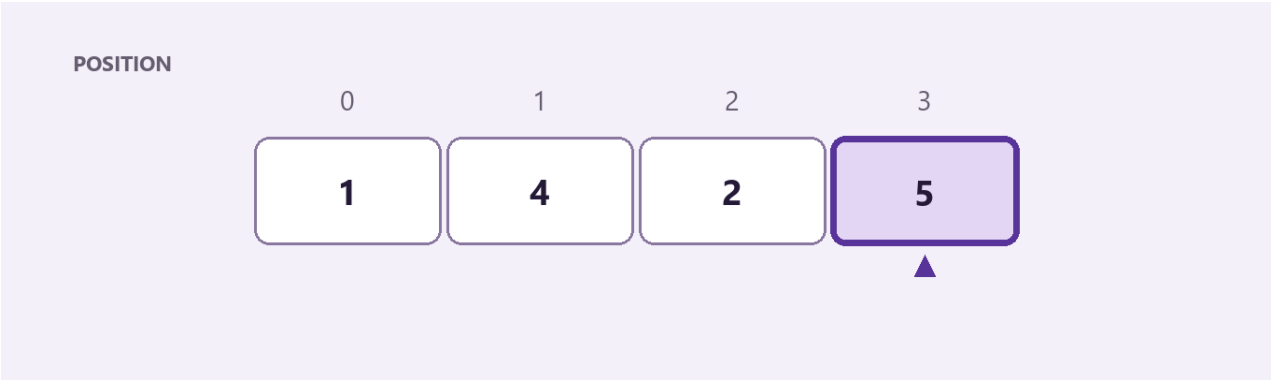


FIG-ALG-06-02 The outlined 5 has reached its final position after pass one on [5, 1, 4, 2]. Look immediately to its left: 4 and 2 still need attention.

Look closer: A pass uses overlapping pairs. On four cards the pairs are positions (0, 1), (1, 2) and (2, 3). Pairing only (0, 1) and (2, 3) leaves a gap where an inverted pair can survive.

Follow the travelling five

Reset the four cards to the original order. Slide a two-cell window one place at a time. Use the updated row for every comparison. Cover the result column until you have moved the cards.

Pair indices	Compared	Row after
0, 1	5 > 1, swap	1 5 4 2
1, 2	5 > 4, swap	1 4 5 2
2, 3	5 > 2, swap	1 4 2 5

Reading the result: Comparing the second pair from the original row would use 1 and 4. By then the 5 has displaced the 1. A trace is a history of changing states, so each line must begin with the result of the preceding line.

Try it yourself: Does one pass move the smallest card to the front in every row?

Check your reasoning: No. In [3, 2, 1] the first pass gives [2, 1, 3]. A small value can move only one place to the left on a left-to-right pass.

Another case: One pass with a quiet pair

Trace a left-to-right pass on [2, 5, 1, 4]. Mark a comparison even when it causes no movement. Which original card reaches its permanent final position, and which inverted pair remains?

Feedback: Compare 2 and 5 and keep them. Swap 5 with 1, then 5 with 4. The result is [2, 1, 4, 5]; 5 has settled at the end while 2 and 1 are inverted. One quiet comparison did not establish that the full row was sorted.

6.3 The boundary moves left

Once the five is parked at the right edge, revisiting it cannot help. Set the next pass's right endpoint one place earlier. This shrinking unsorted region saves comparisons while preserving the largest value at the end of each pass.

For n cards, start end at n - 1. While end > 0, compare positions j and j + 1 for j from zero through end - 1, swapping if A[j] > A[j + 1]. After this inner walk reduce end by one. The two repetitions play different roles: one moves along a pass, the other starts a new pass.

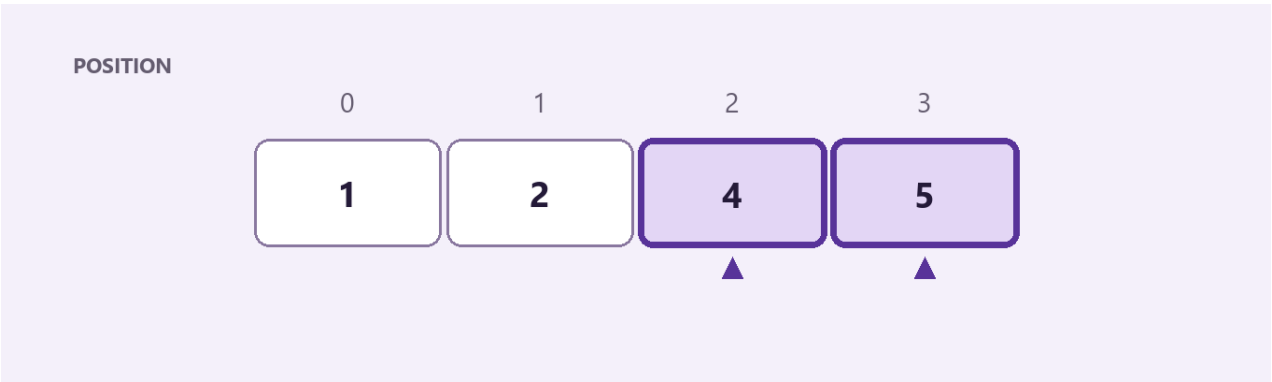


FIG-ALG-06-03 After the second pass both 4 and 5 are settled. A final comparison between 1 and 2 completes the fixed-pass procedure.

Look closer: A fixed pass limit proves that the procedure will stop. The settled-suffix argument explains why the stopping state is ordered. Knowing one of those facts alone would be insufficient to justify the algorithm.

Finish the four-card sort

Continue from [1, 4, 2, 5]. Record comparisons even when the cards stay still. Use a vertical pencil as the boundary of the unfinished part and move it after each pass.

Pass and pair	Action	State
2: 1 and 4	Keep	1 4 2 5
2: 4 and 2	Swap	1 2 4 5
3: 1 and 2	Keep	1 2 4 5

Reading the result: Together the three passes use 3 + 2 + 1 = 6 comparisons and 3 + 1 + 0 = 4 swaps. Counting only visible changes would undercount the work. A correct algorithm may have to establish that a pair is already ordered.

Try it yourself: What happens with zero or one card?

Check your reasoning: end starts below or equal to zero. The outer condition is false, so there is no pair to access. Both rows already meet the ordering rule.

Another case: Removing a comparison removes evidence

A helper shortens the right boundary by two after every pass. Trace [3, 4, 2, 1]. Find a pair the original algorithm would inspect on pass two that the altered one skips.

Feedback: Pass one yields [3, 2, 1, 4]. The altered second pass compares only positions 0 and 1, giving [2, 3, 1, 4]. It skips the needed comparison of 3 and 1 at positions 1 and 2. Settling one largest item permits shortening the region by one.

6.4 Same number, different copies

Two books can share a year yet have different titles. A sorting key is the part of a record that determines its order. Attach a small letter to equal numbers so you can see whether the algorithm changes the order of books with the same key.

A stable sort keeps equal-key items in their original relative order. Ascending Bubble Sort with a strict greater-than swap condition is stable. Two equal neighbours cannot cross unless they exchange places directly, and the condition forbids that exchange.

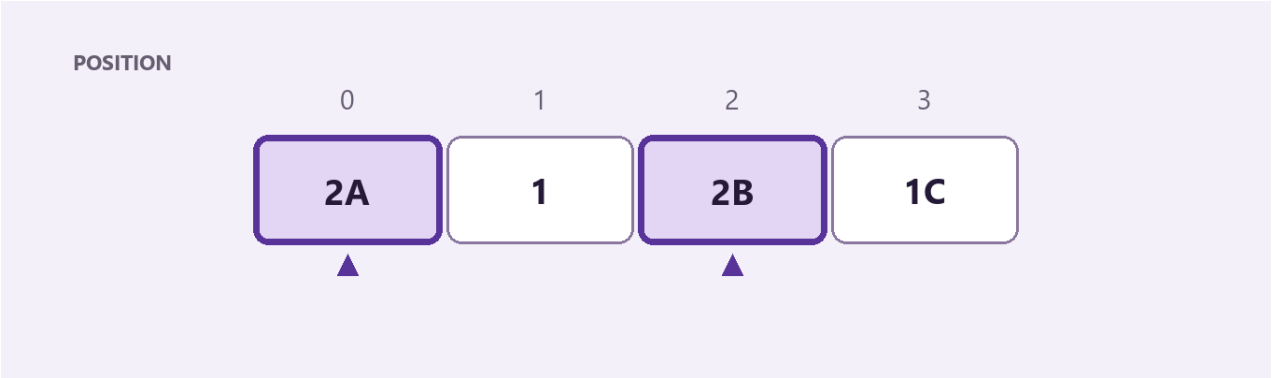


FIG-ALG-06-04 Labels A and B distinguish two records with key 2. Compare only the numeric keys. The letters let you track identities after the movement.

Look closer: A stable sort is useful when the row already has a meaningful order within groups. If returns are entered by arrival time, sorting stably by subject keeps earlier arrivals first within each subject.

Track the identities behind equal keys

Sort records [2A, 1, 2B, 1C], treating 1 and 1C as equal keys. Predict which book with key 2 will appear first. Then change the condition for descending order and watch the equal pair again.

Pass	Ascending state
Start	2A 1 2B 1C
1	1 2A 1C 2B
2	1 1C 2A 2B
3	1 1C 2A 2B

Reading the result: For descending order, swap when the left key is strictly smaller than the right. The result is 2A, 2B, 1, 1C. Sorting ascending and then reversing the entire sequence would also reverse each equal-key group, losing stability.

Try it yourself: Would changing > to ≥ affect the sorted sequence of numbers?

Check your reasoning: The keys can still end up in numeric order, but equal records may exchange identities. On [2A, 2B] one comparison is enough to show the difference.

Another case: Sort a queue of named returns

Three records arrive as Ana/Art, Bo/Science and Cy/Art. A stable subject sort puts Art before Science. Predict the final display order. What evidence would reveal an unstable tie handling even if the displayed subjects are all correct?

Feedback: Stable output is Ana/Art, Cy/Art, Bo/Science. A display with Cy before Ana still has the subjects in sorted order but reverses the Art arrivals. Comparing only subject labels hides this distinction; track a second field such as a name or arrival number.

6.5 Listen for a quiet pass

A helper scans an entire row and never swaps anything. What has the helper learned? Every neighbouring pair in the inspected region was already in the right order. Since the suffix was settled earlier, the full row is now ordered.

An early-stop version resets a marker called swapped to false at the beginning of each pass. Any exchange sets it true. End the sort if it is still false after the complete pass. A single quiet comparison proves very little; the evidence comes from a whole quiet pass.

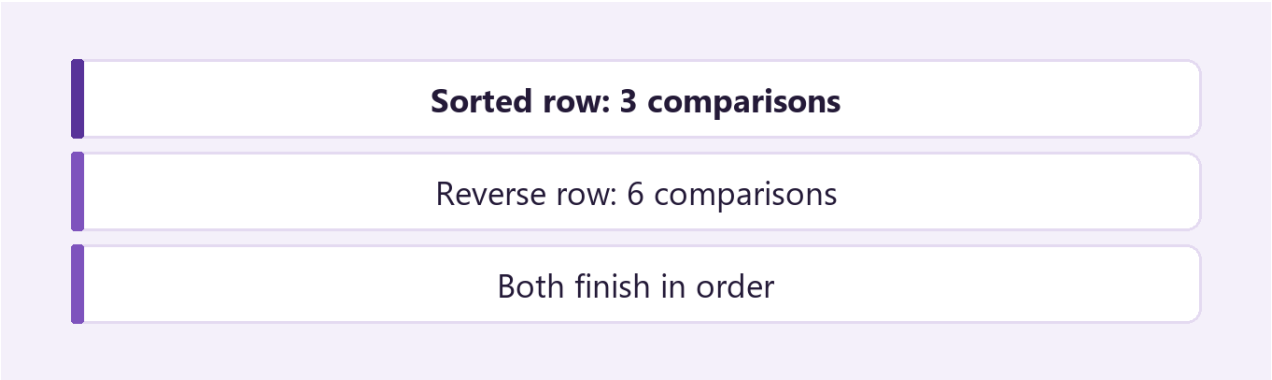


FIG-ALG-06-05 For four cards the first pass has three comparisons. A sorted row finishes that pass with zero swaps. A reverse row requires all three shrinking passes.

Look closer: The early-stop marker records an event over a whole pass. It must remember whether any swap occurred. Assigning it false after every quiet comparison could erase evidence of a swap earlier in the same pass.

Measure what early stopping saves

Compare [1, 2, 3, 4] and [4, 3, 2, 1]. Use the early-stop rule and record a mark for each comparison and a second mark for every exchange. Predict which tally varies most.

Input	Comparisons by pass	Swaps by pass
Sorted	3	0
Reverse	3, 2, 1	3, 2, 1
One inversion [1, 3, 2, 4]	3, 2	1, 0

Reading the result: For the one-inversion row, pass one completes the rearrangement, but the marker records a swap. The second pass supplies the evidence to stop. The number of swaps measures movements; the comparison count measures decisions needed to make or confirm them.

Try it yourself: Why reset the marker before every pass?

Check your reasoning: Once true, a marker left unchanged would remember old work. It would fail to recognize a later quiet pass, losing the early-stop saving.

Another case: A quiet tail can be misleading

On [2, 1, 3, 4], the first comparison swaps and the last two do not. Consider a marker that is replaced with the result of only the latest comparison. Could the same bug stop too early on a less convenient row?

Feedback: Yes. On [3, 2, 1, 4] a pass produces [2, 1, 3, 4]. Its last comparison is quiet even though 2 and 1 remain inverted. The marker must record whether any comparison swapped during the pass, including the earlier ones.

6.6 Which sort did the helper perform?

A different helper scans the whole row to find its smallest card, then exchanges that card with the first position. This is a selection-style pass. It can produce the same final arrangement as Bubble Sort through a different sequence of actions.

The distinction matters when teaching from a demonstration. An ordered result is evidence of an outcome. A trace of the comparisons and moves is evidence of the procedure. The older book-cart activity selected a smallest card; here we use adjacent comparisons to study Bubble Sort.

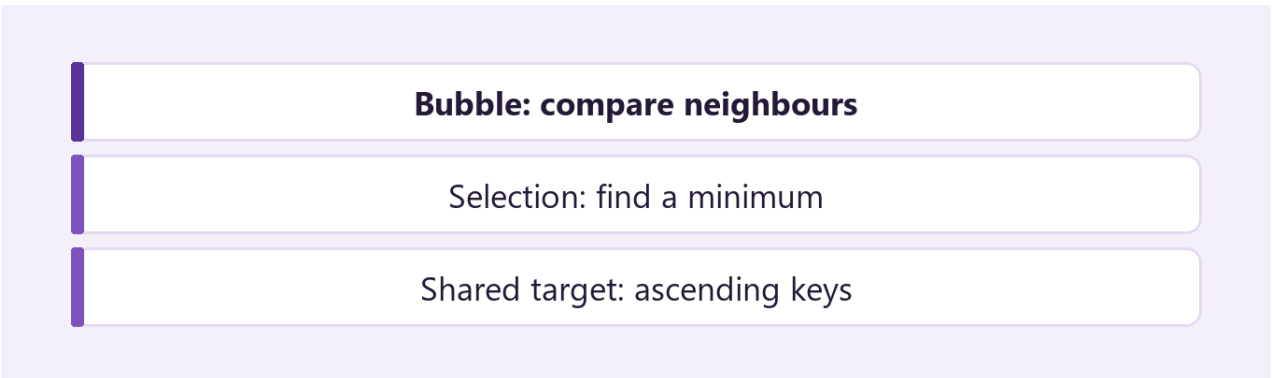


FIG-ALG-06-06 Follow the first movement in each method. A direct exchange between distant positions is decisive evidence against this adjacent-swap Bubble Sort.

Look closer: A first-pass result can sometimes be shared by two methods. Try [2, 1] and see why the short trace is ambiguous. A larger example with its smallest key far from the front is a more revealing comparison.

Reconstruct a method from its trace

Set out [4, 3, 1, 2]. Predict the state after one pass of each method. Run Bubble Sort with a moving pair, then reset and run one minimum-selection pass. Record which final position is guaranteed after each pass.

Method	First pass result	Guaranteed place
Bubble	3 1 2 4	Last holds maximum
Selection	1 3 4 2	First holds minimum

Reading the result: Bubble compares 4 successively with 3, 1 and 2. Selection locates 1 at index 2 before making one exchange with index 0. Neither first pass has promised to sort the entire row. Both methods perform many comparisons as a row grows, motivating a more precise language for work.

Try it yourself: Can one swap tell you which algorithm ran?

Check your reasoning: Sometimes. A distant swap rules out adjacent-only Bubble Sort. A neighbouring swap alone fits both methods. Ask for the comparisons or the next state before drawing a stronger conclusion.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Trace ascending Bubble Sort on [4, 3, 1, 2], showing every comparison and the settled suffix after each pass. Compare its first pass with selecting a minimum and exchanging it with the first item.

Worked guidance

Bubble Sort first produces [3, 4, 1, 2], [3, 1, 4, 2], [3, 1, 2, 4]. The second pass produces [1, 3, 2, 4] then [1, 2, 3, 4]; a final quiet comparison confirms order under this shrinking-pass procedure. Selecting the minimum first gives [1, 3, 4, 2], a different trace.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can trace adjacent swaps, identify a settled suffix and separate comparisons from movements. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

ALG-CH07 / CHAPTER 07

Algorithmic Efficiency and Big O

BY THE END

Choose a unit of work and compare constant, linear, logarithmic and quadratic growth, including extra storage.

Growth labels describe how work scales under assumptions. Count the right operation, name the case, and include preparation or extra storage when the task requires it.

7.1 From ten books to ten thousand

Suppose checking ten book labels is easy. What happens when the library has ten thousand? Measuring today's run might tell you how long that particular machine took. Counting a repeated operation lets you ask a different question: how will the work grow with the collection?

Use n for input size, here the number of books. Count one inspection when a target is compared with a stored key. An unsuccessful linear search inspects each book once. Multiplying the number of books by ten therefore multiplies the inspection count by ten.

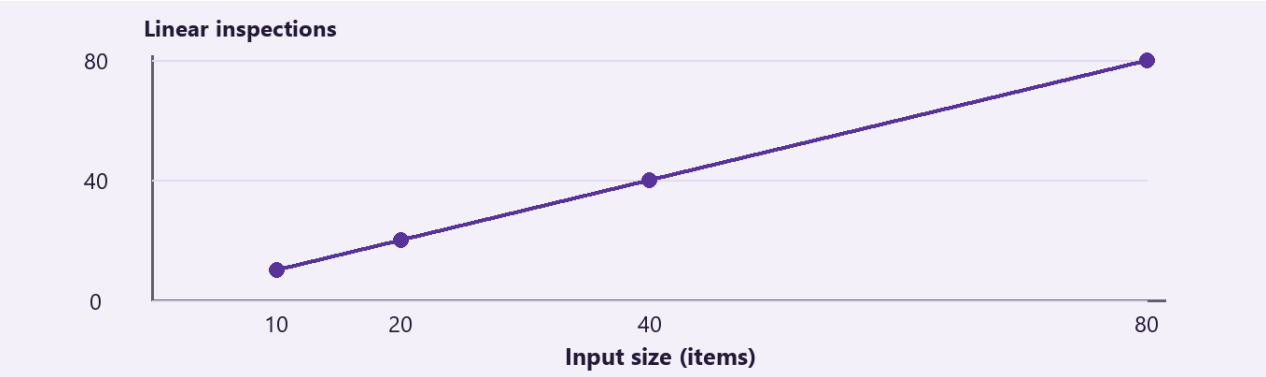


FIG-ALG-07-01 The horizontal axis measures items and the vertical axis measures theoretical inspections for a miss. Doubling the size doubles the count at every marked point.

Look closer: Size can mean different things. In this model it counts records. If titles can be enormously long, comparing two titles may itself require many character checks. Separating record count from key length makes the assumption visible.

Choose one operation to count

Compare scanning for a missing title with reading a shelf label at a known index. Assume direct access to an array slot costs one operation. Predict both columns for a library of 10,000 entries.

Books	Scan for miss	Read known slot
10	10	1
100	100	1
1,000	1,000	1
10,000	10,000	1

Reading the result: The second job starts with the answer to the location question. Reading one known slot and discovering which slot to read are different tasks. Include the work of locating the index whenever your proposed solution has to find it.

Try it yourself: Why might two machines take different times for 100 inspections?

Check your reasoning: A single inspection can involve different processors, storage or key lengths. The count only becomes a time estimate after a cost per inspection has been justified.

Another case: One comparison, many characters

Compare the title strings MAP and MAT one character at a time. Then compare MAP and ZOO. Both jobs compare one pair of keys. Record how many character tests each needs and explain why both counts can be legitimate.

Feedback: MAP and MAT agree on M and A before differing on the third character. MAP and ZOO differ immediately. The costs are three and one character tests, but each is one whole-key comparison. The chosen unit determines which differences the analysis can reveal.

7.2 Whose search counts as typical?

Imagine four catalogue entries, each requested equally often, and suppose every request succeeds. Four possible target positions give four possible inspection counts. The best case is the least work among inputs of this size; the worst case is the most. An average requires a description of which inputs are likely.

For a left-to-right scan the counts are 1, 2, 3 and 4. Add them and divide by four to get 2.5 inspections per request. A single search never inspects half a card. The fraction describes the mean across requests.

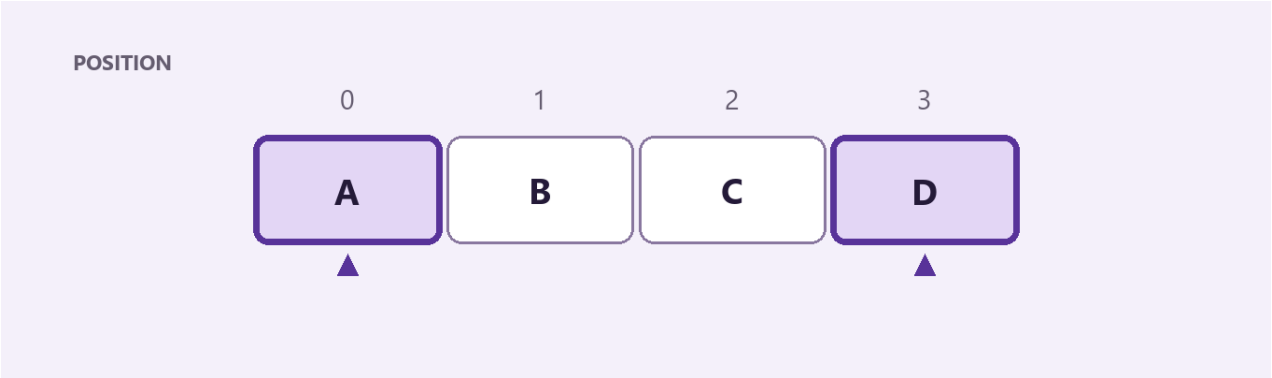


FIG-ALG-07-02 The outlined ends give the best and worst successful positions. The average depends on how often each position is requested, not simply on the endpoints.

Look closer: The mean is a property of both the procedure and the input distribution. If the library changes from mostly common-title requests to mostly missing-title requests, the old mean can cease to describe its workload.

Build an average from stated assumptions

Now change the request pile to A, A, A, D. Use a fresh scan for each request. Record the cost of each scan and compare the observed mean with the equal-likelihood calculation.

Request	Inspections	Running total
A	1	1
A	1	2
A	1	3
D	4	7

Reading the result: The mean is 7 / 4 = 1.75. The worst successful request still costs four. Reporting "average case" without a request model hides an important assumption. Missing targets would add requests that cost four regardless of the position frequencies.

Try it yourself: If all requests are misses, what are the minimum and maximum counts among these runs?

Check your reasoning: Both are four. Restricting the workload to misses narrows the cases being compared, although the algorithm itself has not changed.

Another case: An average with missing requests

Use [A, B, C, D] and requests A, B, X, X, where X is absent. Calculate the mean number of inspections. Which earlier equal-likelihood assumption fails, and which property of the algorithm survives?

Feedback: The counts are 1, 2, 4 and 4, totalling 11 and averaging 2.75. The requests neither succeed uniformly nor choose every stored position equally often. The maximum remains four, since the scan never needs to read more than the whole row once per request.

7.3 Big O as a growth bound

A clerk inspects each card twice and writes three notes at the end. Its count is 2n + 3. Another inspects each card once and writes nothing. Both counts grow in proportion to n as n becomes large. Big O gives an upper growth bound, ignoring constant factors beyond a sufficiently large size.

We write O(n) for a linear upper bound and O(1) for work bounded by a constant independent of n. A very large constant still counts as constant. People often quote the tightest simple bound that fits: although linear work is also bounded by a quadratic curve eventually, “linear” communicates more.

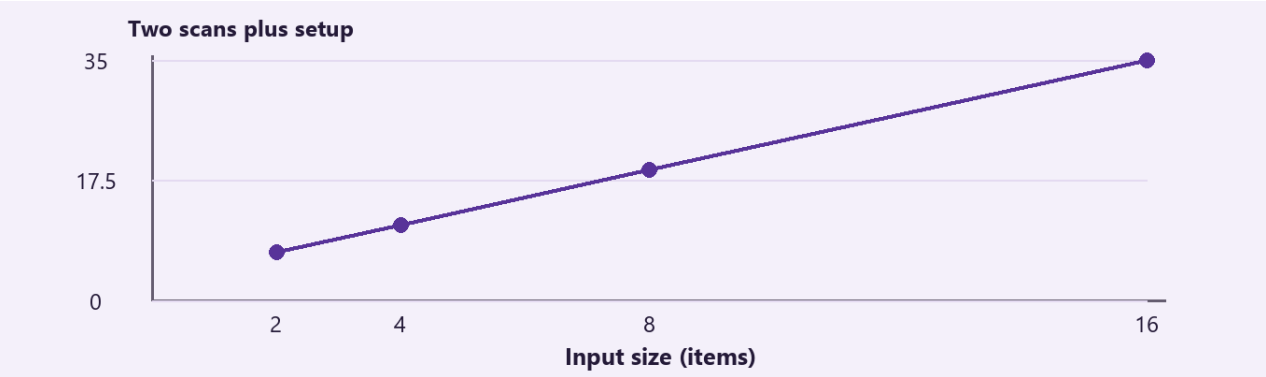


FIG-ALG-07-03 Each point comes from 2n + 3. Increasing n by one adds two inspections in this model. The chart shows calculated work units.

Look closer: For n ≥ 1, 2n + 3 ≤ 5n. This concrete inequality shows how a multiple of n can upper-bound a linear formula. It says nothing about how many seconds either expression represents.

Read the letters without a stopwatch

A third clerk uses a machine that always takes 100 units to open a sealed drawer. Compare that job with the 2n + 3 procedure for several sizes. Decide why the growth label alone cannot settle which is faster at n = 10.

n	2n + 3	Fixed setup
10	23	100
40	83	100
50	103	100

n	2n + 3	Fixed setup
1,000	2,003	100

Reading the result: The linear procedure is cheaper at ten, while the fixed procedure is cheaper at fifty. Big O keeps the difference in growth visible while deliberately discarding details that matter at small sizes. Compare like tasks and stated units when interpreting the numbers.

Try it yourself: Is “at most five indexed reads” constant if the library doubles?

Check your reasoning: Yes, under the direct-access model, provided five is a fixed limit independent of the library size. An unexplained search for each index would add work outside that promise.

Another case: Bound a fixed setup

A machine uses 5n + 20 work units on n cards. For n ≥ 4, find a simple multiple of n that bounds the total. Calculate the actual work for four and eight cards to see why doubling size need not exactly double the total.

Feedback: When n ≥ 4, 20 ≤ 5n, so 5n + 20 ≤ 10n. At four cards the work is 40; at eight it is 60. The constant setup explains the difference from exact doubling, while the linear upper growth bound still holds.

7.4 The cost of halving

Each binary-search inspection removes its midpoint and at least roughly half the candidates. Instead of counting up to n, ask how many times an interval can be halved before it empties. This produces logarithmic growth, written O(log n) for direct midpoint access.

In base two, $\log_2 8 = 3$ because multiplying two by itself three times gives eight. A search on eight entries may still inspect four middle items: a candidate can remain after three halvings. For $n \geq 1$ the maximum number of inspected midpoints is $\text{floor}(\log_2 n) + 1$ with our convention.

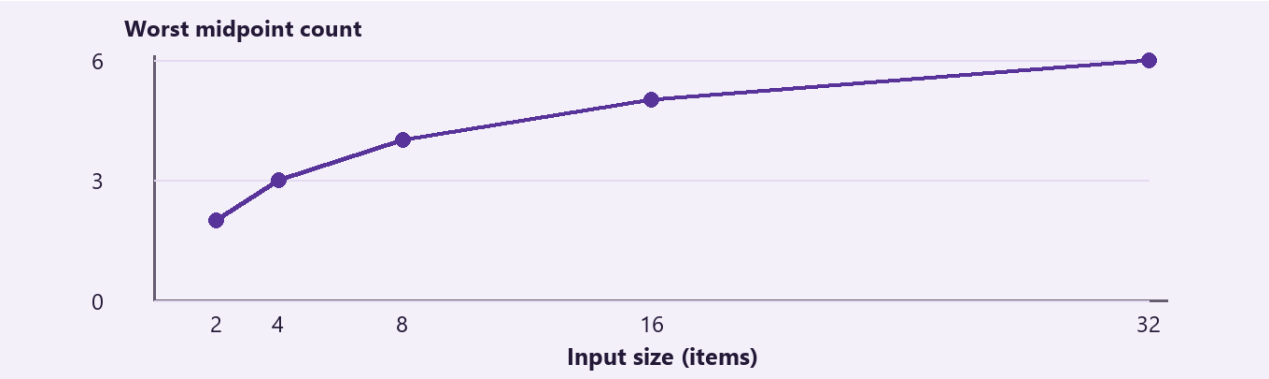


FIG-ALG-07-04 The axis marks are actual collection sizes. Doubling the collection adds just one possible inspection. These are theoretical counts of midpoint visits.

Look closer: Sixteen to thirty-two items doubles the size. Sixteen to seventeen adds one item. Logarithmic growth describes a trend over sizes; every small increase need not increase the worst-case count.

Count how many rounds survive

Start with 16 sorted cards. Imagine always retaining the larger remaining side after a miss. Record the candidate counts just before successive midpoint inspections. Compare the result with a 32-card row.

Inspection	16-card run	32-card run
1	16	32
2	8	16
3	4	8
4	2	4
5	1	2
6	Finished	1

Reading the result: Doubling adds one early stage to the chain. We count one inspection of a key even if actual code uses separate equality and less-than tests for it. An implementation-level comparison count can therefore differ while retaining the same growth.

Try it yourself: Why does shrinking by one item each time give a different result?

Check your reasoning: A chain 16, 15, 14 and so on needs sixteen visits. The amount removed in one step determines how the number of stages grows.

Another case: Count visits on odd sizes

Compare worst-case midpoint inspections for 7, 8 and 9 sorted keys. Build the largest possible remaining side after a failure. Which increase in size adds an inspection and which does not?

Feedback: Seven can shrink through 7, 3, 1, so at most three midpoints are needed. Eight can shrink through 8, 4, 2, 1 and nine through 9, 4, 2, 1. Both need at most four. The cost grows in steps as size increases.

7.5 Why nested walks can be expensive

The first Bubble Sort pass across eight cards compares seven pairs. The next compares six. A fixed-pass sort continues until the last comparison of one pair. The total is a triangular stack of marks. Pairing a short row with a long one reveals its area.

For n entries the total is n(n - 1) / 2. Think of n rows of n - 1 marks split evenly between two matching triangles. This count grows quadratically, O(n²). Two loops alone do not prove a quadratic count. How often the inner body actually runs matters.

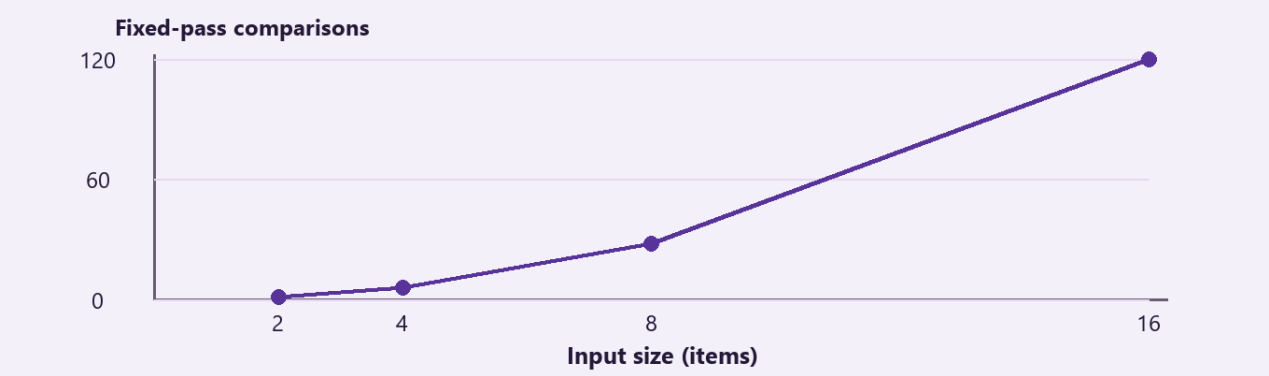


FIG-ALG-07-05 The rising curve is calculated from shrinking passes. At sixteen cards the count is 120, more than four times the 28 comparisons at eight.

Look closer: In the reverse four-card row each of the six pairs of distinct keys is inverted. Every adjacent swap repairs one such pair. This explains why six swaps are necessary even though some particular cards move several times.

Add the shrinking pass lengths

Use dots for pass lengths 5, 4, 3, 2, 1 on six cards. Sum the rows, then compare with a scan and a worst-case binary search. All columns count key inspections or pair comparisons as described.

n	Linear miss	Binary worst	Bubble fixed
4	4	3	6
8	8	4	28
16	16	5	120

Reading the result: For six cards, 5 + 4 + 3 + 2 + 1 = 15, matching 6 × 5 / 2. An early-stop Bubble Sort changes the best case on sorted input to a single linear pass. Its reverse-input worst case remains quadratic.

Try it yourself: If the inside of a nested loop always runs exactly three times per item, what growth results?

Check your reasoning: Three visits per item gives 3n visits, linear growth. The inner loop's length is constant in this example.

Another case: A pair of triangular rows

For five cards, draw pass lengths 4, 3, 2 and 1. For six cards, add a row of length five. How much does the total increase, and why does this matter for a large collection?

Feedback: The total rises from ten to fifteen, an increase of five comparisons. Each extra entry creates an additional potential comparison with every earlier entry in the fixed-pass count. As the size rises, even the price of adding one more item rises.

7.6 What else does the algorithm spend?

A visitor wants a sorted display, but the clerk must preserve the arrival order of the original slips. Sorting the original row would destroy that information. A copy lets the clerk rearrange the display independently. That convenience consumes storage.

Space complexity counts storage as input grows. Auxiliary space counts the extra working storage beyond specified inputs and outputs. Always say which convention you use. A Bubble Sort that swaps items in place needs only a few indices and a temporary item; keeping a full extra copy costs space proportional to n.



FIG-ALG-07-06 Choose based on the required result. If two independent orders must be available, the extra structure supplies information the in-place version would erase.

Look closer: A spare copy of six book titles and six spare integer indices both have length six. Their byte costs may differ substantially. A growth class helps with scaling, while capacity planning often needs sizes in bytes too.

Compare time and extra space

Use six labelled slips in arrival order. Predict the storage you will need to display their sorted order and later reconstruct arrival order. Try sorting one row alone, then try keeping a duplicate row. Record which questions each system can answer.

Design	Extra full-row slots	Recover original order?
In-place sort only	0	Generally no
Sort a separate copy	6	Yes
Keep sorted list of original indices	6 indices	Yes

Reading the result: An index list can be smaller in bytes than copying large book records, although both grow linearly in the number of entries. Time and space are separate axes. A design can improve one while increasing the other.

Try it yourself: How does a known slot provide constant access in the first place?

Check your reasoning: The storage layout lets the machine calculate where that slot begins. Volume II, Chapter 2 builds the idea using equal-width lockers and a starting location.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

A scan costs at most 8 units per request. Preparing a sorted index costs 20 once and each indexed lookup at most 4. Compare one, five and six requests and state a condition that would change your recommendation.

Worked guidance

Scan budgets are 8, 40, 48; indexed budgets are 24, 40, 44. They tie at five requests. Frequent index rebuilding or cheap early scan hits can change the choice. These are declared work budgets, not measured times.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can choose a unit of work and compare constant, linear, logarithmic and quadratic growth, including extra storage. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

Glossary and retrieval cues

Abstraction: A simplified description that keeps the distinctions a task requires. A walking map can omit wall colour while retaining closed paths. [ALG-CH02]

Algorithm: A finite, precise method that solves the specified task for permitted inputs. Searching the cards in order is one method. [ALG-CH01, ALG-CH02]

Assignment: An instruction that stores a value under a name. count <- count + 1 writes a new count. [ALG-CH03]

Binary search: A lookup that uses sorted order to eliminate a side after each failed midpoint inspection. [ALG-CH05]

Boolean: A value that is either true or false. The comparison 2 < 3 yields true. [ALG-CH03]

Bubble Sort: A sorting algorithm that repeatedly compares adjacent entries and exchanges out-of-order pairs. [ALG-CH06]

Complexity: How resource use changes with input size under a stated cost model. Time and extra storage may grow differently. [ALG-CH07]

Condition: A question used to choose a branch or continue repetition. “Are any slips left?” can guard a loop. [ALG-CH03]

Correctness: Meeting the stated task for every permitted input. A route must satisfy its constraints as well as reach its goal. [ALG-CH02]

Decomposition: Dividing a task into smaller jobs whose results fit together. A catalogue lookup supplies a shelf to a route planner. [ALG-CH02]

Flowchart: A diagram showing actions and decisions. Follow the labelled exit from a decision diamond. [ALG-CH03]

Index: A position in an ordered sequence. With zero-based indexing the fourth entry has index 3. [ALG-CH04, DS-CH02]

Input and output: Information received for a job and the result it produces. A book ID can be input to a shelf lookup. [ALG-CH01]

Linear search: A scan that checks entries in sequence. A first-match scan can stop as soon as a comparison succeeds. [ALG-CH04]

Logarithmic growth: Growth where multiplying the problem size by a fixed factor adds roughly a fixed amount of work. [ALG-CH07]

Loop: A structure for repeating instructions. A WHILE loop checks whether another visit is permitted. [ALG-CH03]

Pseudocode: Human-readable instructions for an algorithm. Indentation shows which actions belong to a loop or branch. [ALG-CH03]

Record: A group of fields that belong together. A copy ID, title and shelf can describe one book. [ALG-CH01, DS-CH02]

Sorted: Arranged according to a comparison rule. Binary search requires the rule used for comparing targets to match the ordering. [ALG-CH05, ALG-CH06]

Space complexity: How storage needs grow with input size. Auxiliary space describes extra working storage under a stated convention. [ALG-CH07]

Stable sort: A sorting method that preserves the original relative order of entries with equal keys. [ALG-CH06]

State: The information retained at a particular moment. A trace of the cart can record its current count. [ALG-CH01, ALG-CH03]

Termination: The property of eventually stopping. A strictly shrinking finite search interval supplies a termination argument. [ALG-CH02, ALG-CH05]

Trace: A record of the instructions and states in one run. It can reveal the first step at which a mistake changes the result. [ALG-CH01, ALG-CH03]

Variable: A named place to retain a current value. A running total is a variable used as an accumulator. [ALG-CH03]

Reading, sources and credits

The chapter structure follows the supplied three-volume Visual Edition V1: seven Algorithms chapters, eight Data Structures chapters and ten Connectionist AI chapters. The earlier project manuscript supplies revised stories and worked cases. This edition adds separate teaching sequences, new practice and a larger assessment companion. The page limit was relaxed by the user to allow teaching depth and visuals.

Hello Algo: binary search and Bubble Sort

Hello Algo contributors | ALG-CH03 to ALG-CH07; DS-CH02

A visual textbook provides a second representation of search intervals and algorithm steps.

https://www.hello-algo.com/en/chapter_searching/binary_search/

NIST Dictionary of Algorithms and Data Structures

Paul E. Black / NIST | ALG-CH05 to ALG-CH07

Short formal entries help bridge an intuitive story and a precise algorithm definition.

https://xlinux.nist.gov/dads/HTML/binarySearch.html

MIT 6.006: Introduction to Algorithms, Spring 2020

Erik Demaine, Jason Ku and Justin Solomon / MIT OpenCourseWare | After ALG-CH07 and DS-CH08

A full university course offers lecture notes, lecture videos, practice problems and assignments.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

Web review date: 6 November 2026. Public page review is distinguished from interactive execution in the Learning Materials companion. No external images or large source passages are reproduced. Technical diagrams are original project drawings; the anime illustrations were generated with the built-in image tool using the supplied visual direction. AI assistance supported writing, assessment, art and production.

Project evidence includes earlier source-review and verification records. The approved Markdown Content V2.0, a separate grading rubric and edition-specific instructor feedback were unavailable in that record and remain items for the team to reconcile. This publication does not claim instructor approval, independent student testing or a fresh full viewing of the existing video library. Editable source files and the verification report accompany the PDFs.